
User Manual

for MPC560XB ADC Driver

Document Number: UM13ADCASR4.0 Rev0003R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	13
2.2	Overview.....	14
2.3	About this Manual.....	14
2.4	Acronyms and Definitions.....	15
2.5	Runtime Errors.....	15
2.6	Reference List.....	16
Chapter 3		
ADC Driver		
3.1	Requirements.....	17
3.2	Driver Design Summary.....	17
3.3	Driver Restrictions.....	19
3.4	Deviation from Requirements.....	21
3.5	Software specification.....	22
3.5.1	Define Reference.....	22
3.5.2	Macros Reference.....	22
3.5.2.1	Macro ADC_CTU_START_VALUE.....	22
3.5.2.2	Macro ADC_GROUP_PRIORITY.....	22
3.5.2.3	Macro ADC_GroupInQueue.....	22
3.5.2.4	Macro ADC_GroupsInHwQueue.....	23
3.5.2.5	Macro ADC_GroupTriggerSource.....	23
3.5.2.6	Macro ADC_GroupUnit.....	23
3.5.2.7	Macro ADC_STREAMING_NUM_SAMPLES.....	23
3.5.2.8	Macro ADC_HW_UNITS.....	23
3.5.2.9	Macro CTU_LLD_ClearTRGI.....	24

Section number	Title	Page
3.5.2.10	Macro CTU_LLD_Init_FALSE.....	24
3.5.2.11	Macro CTU_LLD_Init_TRUE.....	24
3.5.2.12	Macro CTU_LLD_ReadEvtCfgRegs.....	24
3.5.2.13	Macro CTU_LLD_ResetEvtCfgRegs.....	25
3.5.2.14	Macro CTU_LLD_ResetPrescalerBits.....	25
3.5.2.15	Macro CTU_LLD_ResetSvrRegs.....	25
3.5.2.16	Macro CTU_LLD_SetCounterStartValue.....	25
3.5.2.17	Macro CTU_LLD_SetEvtCfgReg.....	26
3.5.2.18	Macro CTU_LLD_SetPrescalerValue.....	28
3.5.3	Enum Reference.....	28
3.5.4	Function Reference.....	28
3.5.4.1	Function Adc_DeInit.....	28
3.5.4.2	Function Adc_DisableGroupNotification.....	29
3.5.4.3	Function Adc_DisableHardwareTrigger.....	29
3.5.4.4	Function Adc_EnableGroupNotification.....	30
3.5.4.5	Function Adc_EnableHardwareTrigger.....	30
3.5.4.6	Function Adc_GetGroupStatus.....	31
3.5.4.7	Function Adc_GetStreamLastPointer.....	32
3.5.4.8	Function Adc_GetVersionInfo.....	33
3.5.4.9	Function Adc_Init.....	34
3.5.4.10	Function Adc_ReadGroup.....	34
3.5.4.11	Function Adc_SetupResultBuffer.....	35
3.5.4.12	Function Adc_StartGroupConversion.....	36
3.5.4.13	Function Adc_StopGroupConversion.....	37
3.5.4.14	Function Adc_DMAEndGroupConversion.....	38
3.5.4.15	Function Adc_DMAEndInjectedConv.....	38
3.5.4.16	Function Adc_DMAEndNormalConv.....	39
3.5.4.17	Function Adc_EndCTUConv.....	40
3.5.4.18	Function Adc_EndGroupConversion.....	40

Section number	Title	Page
3.5.4.19	Function Adc_EndInjectedConv.....	41
3.5.4.20	Function Adc_EndNormalConv.....	42
3.5.4.21	Function Adc_EndSwInjectedConv.....	43
3.5.4.22	Function Adc_EndWDGInterrupt.....	43
3.5.4.23	Function Adc_WDGThresholdHandler.....	44
3.5.4.24	Function Adc_ConfigureThreshold.....	45
3.5.4.25	Function Adc_DisableChannel.....	45
3.5.4.26	Function Adc_DisableCtuTrigger.....	46
3.5.4.27	Function Adc_EnableChannel.....	46
3.5.4.28	Function Adc_EnableCtuTrigger.....	47
3.5.4.29	Function Adc_GetInjectedConversionStatus.....	48
3.5.4.30	Function Adc_HwResultReadGroup.....	48
3.5.4.31	Function Adc_SetClockMode.....	49
3.5.4.32	Function Adc_SetMode.....	50
3.5.5	Structs Reference.....	51
3.5.5.1	Structure Adc_ConfigType.....	51
3.5.5.2	Structure Adc_GroupStatusType.....	52
3.5.5.3	Structure Adc_LLD_ChannelConfiguration.....	53
3.5.5.4	Structure Adc_LLD_GroupConfiguration.....	54
3.5.5.5	Structure Adc_LLD_HWUnitConfiguration.....	56
3.5.5.6	Structure Adc_LLD_MhtChannelGroupType.....	57
3.5.5.7	Structure Adc_LLD_MultiConfig.....	58
3.5.5.8	Structure Adc_UnitStatusType.....	59
3.5.6	Types Reference.....	60
3.5.7	Variables Reference.....	60
3.6	Symbolic Names Disclaimer.....	60

Chapter 4 Tresos Configuration Plug-in

4.1	Configuration elements of Adc.....	61
-----	------------------------------------	----

Section number	Title	Page
4.2	Form IMPLEMENTATION_CONFIG_VARIANT.....	61
4.3	Form AdcGeneral.....	62
4.3.1	AdcDeInitApi (AdcGeneral).....	62
4.3.2	AdcDevErrorDetect (AdcGeneral).....	63
4.3.3	AdcEnableLimitCheck (AdcGeneral).....	63
4.3.4	AdcEnableQueuing (AdcGeneral).....	63
4.3.5	AdcEnableStartStopGroupApi (AdcGeneral).....	64
4.3.6	AdcGrpNotifCapability (AdcGeneral).....	64
4.3.7	AdcHwTriggerApi (AdcGeneral).....	65
4.3.8	AdcPriorityImplementation (AdcGeneral).....	65
4.3.9	AdcReadGroupApi (AdcGeneral).....	65
4.3.10	AdcResultAlignment (AdcGeneral).....	66
4.3.11	AdcVersionInfoApi (AdcGeneral).....	66
4.3.12	AdcConvTimeOnce (AdcGeneral).....	67
4.3.13	AdcPreSamplingOnce (AdcGeneral).....	67
4.3.14	AdcWdgThresholdOnce (AdcGeneral).....	67
4.3.15	AdcSetOnceRegisters (AdcGeneral).....	68
4.3.16	ADCDirectDMAResultBufferCopy (AdcGeneral).....	68
4.3.17	AdcTimeout (AdcGeneral).....	69
4.3.18	AdcMuxDelaySupport (AdcGeneral).....	69
4.3.19	AdcMuxDelaySupportPerGroup (AdcGeneral).....	70
4.4	Form AdcPublishedInformation.....	70
4.4.1	AdcChannelValueSigned (AdcPublishedInformation).....	71
4.4.2	AdcGroupFirstChannelFixed (AdcPublishedInformation).....	71
4.4.3	AdcMaxChannelResolution (AdcPublishedInformation).....	71
4.5	Form CommonPublishedInformation.....	72
4.5.1	ArReleaseMajorVersion (CommonPublishedInformation).....	72
4.5.2	ArReleaseMinorVersion (CommonPublishedInformation).....	73
4.5.3	ArReleaseRevisionVersion (CommonPublishedInformation).....	73

Section number	Title	Page
4.5.4	ModuleId (CommonPublishedInformation).....	74
4.5.5	SwMajorVersion (CommonPublishedInformation).....	74
4.5.6	SwMinorVersion (CommonPublishedInformation).....	75
4.5.7	SwPatchVersion (CommonPublishedInformation).....	75
4.5.8	VendorApiInfix (CommonPublishedInformation).....	75
4.5.9	VendorId (CommonPublishedInformation).....	76
4.6	Form NonAutosar.....	76
4.6.1	AdcEnableCtuTrigNonAutosarApi (NonAutosar).....	77
4.6.2	AdcSetModeApi (NonAutosar).....	77
4.6.3	AdcEnableDualClockMode (NonAutosar).....	78
4.6.4	AdcEnableGroupDependentChannelNames (NonAutosar).....	78
4.6.5	AdcBypassConsistencyLoop (NonAutosar).....	79
4.6.6	AdcEnableChDisableChApi (NonAutosar).....	79
4.6.7	AdcGetInjectedConvStatusApi (NonAutosar).....	80
4.6.8	AdcErrataInterruptedNormalGroup (NonAutosar).....	80
4.6.9	AdcErrataOptimizedSwAbort (NonAutosar).....	81
4.6.10	AdcDisableDemReportErrorStatus (NonAutosar).....	81
4.6.11	AdcEnableThresholdConfiguration (NonAutosar).....	82
4.7	Form AdcDemEventParameterRefs.....	82
4.7.1	ADC_E_TIMEOUT (AdcDemEventParameterRefs).....	83
4.8	Form AdcConfigSet.....	83
4.8.1	Form AdcGeneric.....	84
4.8.1.1	AdcPriorityQueueMaxDepth (AdcGeneric).....	84
4.8.1.2	AdcTransferType (AdcGeneric).....	85
4.8.2	Form AdcHwUnit.....	85
4.8.2.1	AdcClockSource (AdcHwUnit).....	86
4.8.2.2	AdcHwUnitId (AdcHwUnit).....	86
4.8.2.3	AdcPrescale (AdcHwUnit).....	87
4.8.2.4	AdcPowerDownDelay (AdcHwUnit).....	87

Section number	Title	Page
4.8.2.5	AdcAltPowerDownDelay (AdcHwUnit).....	88
4.8.2.6	AdcMuxDelay (AdcHwUnit).....	88
4.8.2.7	AdcAutoClockOff (AdcHwUnit).....	89
4.8.2.8	AdcBypassSampling (AdcHwUnit).....	89
4.8.2.9	AdcPresamplingEnableSignal (AdcHwUnit).....	90
4.8.2.10	Form AdcNormalAltConvTimings.....	90
4.8.2.10.1	AdcLatchingDurationNormal (AdcNormalAltConvTimings).....	91
4.8.2.10.2	AdcComparisonDurationNormal (AdcNormalAltConvTimings).....	91
4.8.2.10.3	AdcSamplingDurationNormal (AdcNormalAltConvTimings).....	91
4.8.2.10.4	AdcLatchingDurationAlt (AdcNormalAltConvTimings).....	92
4.8.2.10.5	AdcComparisonDurationAlt (AdcNormalAltConvTimings).....	92
4.8.2.10.6	AdcSamplingDurationAlt (AdcNormalAltConvTimings).....	93
4.8.2.11	Form AdcChannel.....	93
4.8.2.11.1	AdcChannelConvTime (AdcChannel).....	94
4.8.2.11.2	AdcChannelHighLimit (AdcChannel).....	95
4.8.2.11.3	AdcChannelId (AdcChannel).....	95
4.8.2.11.4	AdcHwChannel (AdcChannel).....	95
4.8.2.11.5	AdcChannelLimitCheck (AdcChannel).....	96
4.8.2.11.6	AdcChannelLowLimit (AdcChannel).....	96
4.8.2.11.7	AdcChannelRangeSelect (AdcChannel).....	97
4.8.2.11.8	AdcChannelRefVoltsrcHigh (AdcChannel).....	97
4.8.2.11.9	AdcChannelRefVoltsrcLow (AdcChannel).....	97
4.8.2.11.10	AdcChannelResolution (AdcChannel).....	98
4.8.2.11.11	AdcChannelSampTime (AdcChannel).....	98
4.8.2.11.12	AdcEnablePresampling (AdcChannel).....	99
4.8.2.11.13	Form AdcThresholdControl.....	99
4.8.2.11.13.1	AdcEnableThresholds (AdcThresholdControl).....	100
4.8.2.11.13.2	AdcThresholdControlRegister (AdcThresholdControl).....	100
4.8.2.11.13.3	AdcHighThreshold (AdcThresholdControl).....	101

Section number	Title	Page
4.8.2.11.13.4	AdcLowThreshold (AdcThresholdControl).....	101
4.8.2.11.13.5	AdcWdogNotification (AdcThresholdControl).....	102
4.8.2.12	Form AdcGroup.....	102
4.8.2.12.1	AdcGroupAccessMode (AdcGroup).....	103
4.8.2.12.2	AdcGroupConversionMode (AdcGroup).....	104
4.8.2.12.3	AdcGroupConversionType (AdcGroup).....	104
4.8.2.12.4	AdcGroupId (AdcGroup).....	104
4.8.2.12.5	AdcGroupPriority (AdcGroup).....	105
4.8.2.12.6	AdcGroupReplacement (AdcGroup).....	105
4.8.2.12.7	AdcGroupTriggSrc (AdcGroup).....	106
4.8.2.12.8	AdcHwTrigSignal (AdcGroup).....	106
4.8.2.12.9	AdcHwTrigTimer (AdcGroup).....	106
4.8.2.12.10	AdcNotification (AdcGroup).....	107
4.8.2.12.11	AdcStreamingBufferMode (AdcGroup).....	107
4.8.2.12.12	AdcStreamingNumSamples (AdcGroup).....	108
4.8.2.12.13	AdcEnableChDisableChGroup (AdcGroup).....	108
4.8.2.12.14	AdcMuxDelayNeeded (AdcGroup).....	108
4.8.2.12.15	AdcWithoutInterrupts (AdcGroup).....	109
4.8.2.12.16	AdcMultipleHardwareTriggerGroup (AdcGroup).....	109
4.8.2.12.17	Form AdcGroupDefinition.....	110
4.8.2.12.17.1	AdcGroupDefinition (AdcGroupDefinition).....	110
4.8.2.12.18	Form AdcGroupConversionConfiguration.....	111
4.8.2.12.18.1	AdcLatchingDuration (AdcGroupConversionConfiguration).....	111
4.8.2.12.18.2	AdcComparisonDuration (AdcGroupConversionConfiguration).....	111
4.8.2.12.18.3	AdcSamplingDuration (AdcGroupConversionConfiguration).....	112
4.8.2.12.19	Form AdcHwTrig.....	113
4.8.2.12.19.1	AdcHwTrigSrc (AdcHwTrig).....	113



Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	08-Feb-2012	Massimo De Martino	Initial Draft for Beta 0.9.0
1.1	14-May-2013	Indranil Bhattacharya	Updated for changes in RTM 1.0.0
1.2	02-June-2014	Franck Thebault	Updated for changes in RTM 1.0.1



Chapter 2

Introduction

This User Manual describes Freescale Semiconductor AUTOSAR Analogue to Digital Converter (ADC) for MPC5607B , MPC5604B , 5606B and 5602D devices .

AUTOSAR ADC driver configuration parameters and deviations from the specification are described in ADC Driver chapter of this document. AUTOSAR ADC driver requirements and APIs are described in the AUTOSAR ADC driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor .

Table 2-1. MPC5607B Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100
-------------------------	--

Table 2-1. MPC5607B Derivatives

	mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
--	--

All of the above microcontroller devices are collectively named as MPC5607B .

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
ADC	Analog to Digital Converter
API	Application Programming Interface
ASM	Assembler
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Runtime Errors

The driver generates the following DEM errors at runtime.

Table 2-3. Runtime Errors

Function	Error Code	Condition triggering the error
Adc_Init()	Adc_E_Timeout	Timeout expired. ADC hardware is not entered in power down state.

Table continues on the next page...

Table 2-3. Runtime Errors (continued)

Function	Error Code	Condition triggering the error
Adc_Init()	Adc_E_Timeout	Timeout expired. ADC hardware is not entered in idle state.
Adc_StartGroupConversion()	Adc_E_Timeout	Timeout expired. Ongoing conversion is not aborted.
Adc_StopGroupConversion()	Adc_E_Timeout	Timeout expired. Ongoing conversion is not aborted.
Adc_EnableHardwareTrigger()	Adc_E_Timeout	Timeout expired. ADC hardware is not entered in power down state.
Adc_EnableCtuTrigger()	Adc_E_Timeout	Timeout expired. ADC hardware is not entered in power down state.
Adc_SetMode()	Adc_E_Timeout	Timeout expired. ADC hardware is not entered in power down state.
Adc_SetMode()	Adc_E_Timeout	Timeout expired. ADC hardware is not entered in idle state.
Adc_EndNormalConv()	Adc_E_Timeout	Timeout expired. Ongoing conversion is not aborted.
Adc_DMAEndNormalConv()	Adc_E_Timeout	Timeout expired. Ongoing conversion is not aborted.
Adc_EnableHardwareTrigger()	Adc_E_Timeout	Timeout expired. double buffered registers are not updated.
Adc_DisableHardwareTrigger()	Adc_E_Timeout	Timeout expired. double buffered registers are not updated.

2.6 Reference List

Table 2-4. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003ADC Driver Software Specification Document.	V4.2.0
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013

Chapter 3

ADC Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003ADC Driver Software Specification document (See Table [Reference List](#)).

3.2 Driver Design Summary

The ADC Driver initializes and controls the internal Analogue to Digital Converter Unit(s) of the microcontroller. It provides services to start and stop a conversion respectively to enable and disable the trigger source for a conversion. Furthermore it provides services to enable and disable a notification mechanism and routines to query the status and result of a conversion. The ADC Driver shall work on so called ADC Channels. An ADC channel combines an analogue input pin, the needed ADC circuitry itself and a conversion result register into an entity that can be individually controlled and accessed via the ADC Driver. The driver provides a service for Streaming management results and for De-Initialization of circuits.

Moreover the following ADC units hardware channels will be used:

- **MPC5607B device families at 208 | 176 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15; ANS_0:ANS_27; ANX_0:ANX_31;
 - Adc Physical Channels for ADC HW Unit 1: ANP_0:ANP_15; ANS_0:ANS_7;
- **MPC5607B device families at 144 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15; ANS_0:ANS_2;ANS_5:ANS_15; ANX_0:ANX_31;
 - Adc Physical Channels for ADC HW Unit 1: ANP_0:ANP_15; ANS_0:ANS_7;
- **MPC5607B device families at 100 pins:**

- Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15;
ANS_0:ANS_2;ANS_5:ANS_7 ANX_0:ANX_31;
- Adc Physical Channels for ADC HW Unit 1: ANP_0:ANP_15; ANS_0:ANS_7;
- **MPC5604B device families at 208 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15;
ANS_0:ANS_15; ANX_0:ANX_31;
- **MPC5604B device families at 144 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15;
ANS_0:ANS_15; ANX_0:ANX_31;
- **MPC5604B device families at 100 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15; ANS_0:ANS_7;
ANX_0:ANX_31;
- **MPC5604B (B) device families at 64 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_3; ANS_0:ANS_3;
ANX_0:ANX_3;ANX_8:ANX_11;ANX_16:ANX_19;ANX_24:ANX_27;
- **MPC5604B (C) device families at 64 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_3; ANS_0:ANS_3;
ANX_16:ANX_19;ANX_24:ANX_27;
- **5606B device families at 176 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15;
ANS_0:ANS_27; ANX_0:ANX_31;
 - Adc Physical Channels for ADC HW Unit 1: ANP_0:ANP_15; ANS_0:ANS_7;
- **5606B device families at 144 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15;
ANS_0:ANS_2;ANS_5:ANS_15; ANX_0:ANX_31;
 - Adc Physical Channels for ADC HW Unit 1: ANP_0:ANP_15; ANS_0:ANS_7;
- **5606B device families at 100 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_15;
ANS_0:ANS_2;ANS_5:ANS_7 ANX_0:ANX_31;
 - Adc Physical Channels for ADC HW Unit 1: ANP_0:ANP_15; ANS_0:ANS_7;
- **5602D device families at 100 pins:**
 - Adc Physical Channels for ADC HW Unit 0 (ADC1): ANP_0:ANP_15;
ANS_0:ANS_12; ANX_0:ANX_31;
- **5602D device families at 64 pins:**
 - Adc Physical Channels for ADC HW Unit 0: ANP_0:ANP_3; ANS_0:ANS_6;
ANX_0:ANX_31;

3.3 Driver Restrictions

In the Driver there are the following restrictions.

For injected groups:

- the groups configured as injected must have the same "common" configuration as the groups that will be interrupted;
- the injected group (Sw/Hw) must not contain the channels that are part of the group that will be interrupted (**intersection of the channels sets must be avoided**).

For NonAutosar function Adc_SetClockMode():

- the Non Autosar function Adc_SetClockMode() works on the ADCCLKSET bit and in order to divide by one or by two, set this bit if the funciotn is called in *Alternate Mode* or in *Normal Mode* respectively. Moreover the ADCCLKSEL bit can be changed only in *ADC Power Down Mode*. The functional clock range are in the [Table 3-1](#) and in [Table 3-2](#);
- the Non Autosar function Adc_SetClockMode() works on the both ADC units (if enabled) using the same pamameters.

Usage of Adc_SetClockMode(), Adc_SetMode() APIs

- The Adc_SetClockMode() will work only if ADC driver is initialized and Adc_SetMode(ADC_POWER_DOWN_MODE) has been called (in that order).
- Once Adc_SetClockMode() has successfully returned Adc_SetMode(ADC_NORMAL_MODE) should be called before starting any group conversions.
- It should be ensured that no conversions are on-going when calling the Adc_SetMode().

Normal conversions interrupted by Injected Conversion

If an injected conversion is triggered when a normal conversion is ongoing, and the aborted normal channel is contained in the injected group, the conversion of the aborted channel will not be restored in the normal conversion

Example: Nch1-> Nch2(aborted)->Jch1 -> Jch2> Jch3 -> Nch3->Nch4 (Nch2 not restorted)

Note that ch2 is both in the normal conversion group and in the injected group, So data for Nch2 will be available from the injected chain instead.

Also note that in the example above the injected chain will overwrite the conversion results of Nch1

Software Injected conversions and CTU triggered conversions not allowed in parallel

Driver Restrictions

When an Injected conversion is requested during CTU conversion, the injected conversion is discarded and the MCR[JSTART] is immediately reset.

It can be thus NEVER guaranteed that when a CTU triggered group is enabled, a parallel SW Injected group will be completed.

A CTU triggered group and a SW injected group should never be enabled / executed simultaneously. The SW injected group should be executed only when no CTU triggered group is enabled and vice-versa.

External channels ANX_xxx cannot be triggered by CTU.

If using XPC560xD device, note that CTU_EVTCTFGRx[CHANNEL_VALUE] bitfield is 6 bit wide and can select ADC channels within

the range 0-63, hence external channels CH[64-95], cannot be selected.

PIT events cannot be used to trigger ADC conversion incase BCTU runs on divided system clock.

If using XPC560xD device, note that If CTU operates on divided system clock (from MCU), events from PIT timer cannot be used to trigger ADC conversion. In this case BCTU fails to latch the channel number to be converted.

So CTU will send the conversion request to ADC but the channel number will be corresponding to previous non-PIT conversion

request or 0th channel in case no previous non-PIT event has occurred.

So if using PIT events to trigger CTU and then ADC, please select the undivided system clock frequency for CTU in the MCU

configuration.

DMA transfer type for HW triggered groups (non-CTU)

- **If DMA is used as `AdcTransferType`, user must not run a SW and a HW group at the same time on the same HW unit because the same DMA channel will be used for both.**
- If DMA is set for `AdcTransferType`, `AdcWithoutInterrupts` should be selected as false otherwise, just like done currently for SW groups, DMA will not be configured and it will be user responsibility to read the results from registers directly (using `Adc_ReadGroup()`).

NonAutosar Bypass Consistency Loop parameter:

- `AdcBypassConsistencyLoop` parameter can be checked if and only if there isn't Priority and the Queue is disabled.

Table 3-1. Clock range for Bolero 1.5M and 256K

Vref(V)	Clock(MHz) for ADC 0(Only Bolero 1.5M)	Clock(MHz) for ADC 1
3.3V	6, 7, 8, 16, 32(Only Bolero 1.5M)	4, 5, 6, 7, 8, 16, 20
5.0V	6, 7, 8, 16, 32(Only Bolero 1.5M)	4, 5, 6, 6, 7, 8, 16, 32

Table 3-2. Clock range for Bolero 512K

Vref(V)	Clock(MHz) for ADC 0
5.0V/3.3V	6, 7, 8, 16, 32

3.4 Deviation from Requirements

The driver deviates from the AUTOSAR ADC Driver software specification in some places. The table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the ADC Driver. Table [Table 3-3](#) provides Status column description.

Table 3-3. Deviations Status Column Description

Term	Definition
N/A	Not available
N/T	Not testable
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the driver.

Table 3-4. ADC Deviations Table

Requirement	Status	Description	Notes
ADC345	N/I	The ADC module priority mechanism shall allow suspending and resuming of channel group conversion.	For this version has been implemented the abort/restart mechanism.
ADC341, ADC339, ADC337	N/A	Priority mechanism supported by hardware.	Hw queue are not supported by hardware.
ADC319	N/F	In streaming access mode the result buffer shall have m*n elements, where n is the numbers of channels belonging to the group,	The implementation is as per n*m as shown in the Figure 2 of the 4.0 Rev0003ADC (version V4.2.0) [Reference List]: Example for Group

Table 3-4. ADC Deviations Table

Requirement	Status	Description	Notes
		m the number of the sample acquired per channel.	and Result Buffer configuration of Autosar SWS but autosar ADC319 requirement is inconsistent with this figure.

3.5 Software specification

The following sections contains driver software specifications.

3.5.1 Define Reference

Constants supported by the ADC driver.

3.5.2 Macros Reference

Macros supported by the ADC driver.

3.5.2.1 Macro ADC_CTU_START_VALUE

macros to simplify access to structures.

Prototype: `ADC_CTU_START_VALUE (num)`

3.5.2.2 Macro ADC_GROUP_PRIORITY

Adc group priority.

Prototype: `ADC_GROUP_PRIORITY (Priority)`

3.5.2.3 Macro ADC_GroupInQueue

The group at position p in queue of given hardware unit.

Prototype: `ADC_GroupInQueue (unit, p)`

3.5.2.4 Macro ADC_GroupsInHwQueue

Number of hardware groups in queue for hardware unit.

Prototype: `ADC_GroupsInHwQueue(unit)`

3.5.2.5 Macro ADC_GroupTriggerSource

Adc hardware trigger source configured for the group.

Prototype: `ADC_GroupTriggerSource(group, pos)`

3.5.2.6 Macro ADC_GroupUnit

Configured hardware unit of the group.

Prototype: `ADC_GroupUnit(group)`

3.5.2.7 Macro ADC_STREAMING_NUM_SAMPLES

macros to simplify access to structures.

Details:

Streaming number of samples

Prototype: `ADC_STREAMING_NUM_SAMPLES(num)`

3.5.2.8 Macro ADC_HW_UNITS

ADC Hardware unit macro for register access.

Implements: DADC04057

Note

lint -save -e961

Violates: Use of function like macro.

Prototype: `ADC_HW_UNITS(unit)`

3.5.2.9 Macro **CTU_LLD_ClearTRGI**

This macro clears the TRGIE bit in the CTU_CSR register.

Details:

Clears the TRGIE field in the CTU_CSR register in order to ack the interrupt.

Implements:

Prototype: `CTU_LLD_ClearTRGI()`

3.5.2.10 Macro **CTU_LLD_Init_FALSE**

Prototype: `CTU_LLD_Init_FALSE()`

3.5.2.11 Macro **CTU_LLD_Init_TRUE**

This macro initializes the CTU hardware module.

Details:

Sets TRGIEN bit, in conformance with the parameter, from the CTU_CSR register.

Implements:

Prototype: `CTU_LLD_Init_TRUE()`

3.5.2.12 Macro **CTU_LLD_ReadEvtCfgRegs**

This macro read the CTU hardware module CTU_EVTCFGRx registers.

Details:

Read the CTU_EVTCFGRx register and mask with the Trigger Mask bit.

Implements:

Prototype: `CTU_LLD_ReadEvtCfgRegs(TriggSrc)`

3.5.2.13 Macro CTU_LLD_ResetEvtCfgRegs

This macro initializes the CTU hardware module CTU_EVTCFGRx registers.

Details:

Resets all CTU_EVTCFGRx registers (write 0 for reset value).

Implements:

Prototype: `CTU_LLD_ResetEvtCfgRegs()`

3.5.2.14 Macro CTU_LLD_ResetPrescalerBits

This macro initializes the CTU hardware module.

Details:

Resets PRESC_CONF bits from the CTU_CSR register (write 0 for reset value).

Implements:

Prototype: `CTU_LLD_ResetPrescalerBits()`

3.5.2.15 Macro CTU_LLD_ResetSvrRegs

This macro initializes the CTU hardware module CTU_SVRx registers.

Details:

Resets all CTU_SVRx registers (write 0 for reset value).

Implements:

Prototype: `CTU_LLD_ResetSvrRegs()`

3.5.2.16 Macro CTU_LLD_SetCounterStartValue

This macro initializes the CTU hardware module CTU_SVRx registers.

Details:

Sets the CTU_SVRx register, in conformance with the parameter.

Implements:

Prototype: `CTU_LLD_SetCounterStartValue (Index, CounterStartValue)`

3.5.2.17 Macro CTU_LLD_SetEvtCfgReg

Configures the CTU hardware module CTU_EVT_CFGRx register with the desired values.

Details:

Updates all fields in the CTU_EVT_CFGRx register (the register is specified by the TriggerSource parameter) AdcHwUnitSelector is uint8 because on some future platforms can be more than 2 ADC hw units.

- N.B.: Bolero 256k-128k has 1 ADC hw unit.

Implements:

Prototype:

`CTU_LLD_SetEvtCfgReg (TriggerSource, TriggerMask, ClearFlag, CountGroup, DelayIndex, AdcChannelID)`

Table 3-5. CTU_LLD_SetEvtCfgReg Arguments

Name	Direction	Description
TriggerSource	input	Specifies the trigger source index (ch0-eMIOs0, ch32-PIT_3).
TriggerMask	input	The value of the Trigger Mask field: 0: Trigger masked 1: Trigger enabled.
ClearFlag	input	The value of the CLR_FLAG field: To provide flag_ack through software 1: Flag_ack is forced to '1' for the particular event 0:

Table continues on the next page...

**Table 3-5. CTU_LLD_SetEvtCfgReg Arguments
(continued)**

Name	Direction	Description
		Flag_ack is dependent on flag servicing This bit implementation is generic based and implemented only for inputs mapped to PIT event flags.
CountGroup	input	The value of the COUNT_GROUP field: 00: Counter 0 is associated with the particular event 01: Counter 1 is associated with the particular event 10: Counter 2 is associated with the particular event 11: Counter 3 is associated with the particular event.
DelayIndex	input	The value of the DELAY_INDEX field: 000: No delay is provided 001: Counter is loaded with value stored in CTU_SVR1 010: Counter is loaded with value stored in CTU_SVR2 011: Counter is loaded with value stored in CTU_SVR3 100: Counter is loaded with value stored in CTU_SVR4 101: Counter is loaded with value stored in CTU_SVR5 110: Counter is loaded with value stored in CTU_SVR6 111: Counter is loaded with value stored in CTU_SVR7.
AdcChannelID	input	The ID of the ADC channel (in order to implement the ADC channel mapping over the CTU channels) CHANNEL_VALUE[x:0] (the precision is platform dependent) These bits provide the ADC channel number to be converted. Valid values are 0b0 to 0b1011111 (decimal 95).

3.5.2.18 Macro CTU_LLD_SetPrescalerValue

This macro initializes the CTU hardware module.

Details:

Sets PRESC_CONF bits, in conformance with the parameter, from the CTU_CSR register.

Implements:

Prototype: CTU_LLD_SetPrescalerValue(PrescConfig)

3.5.3 Enum Reference

Enumeration of all constants supported by the ADC driver.

3.5.4 Function Reference

Functions supported by the ADC driver.

3.5.4.1 Function Adc_DeInit

Returns all ADC HW Units to a state comparable to their power on reset state.

Details:

Returns all ADC HW Units to a state comparable to their power on reset state, and de-initialize the ADC MCAL driver.

Return: void.

Implements: DADC03014.

Note

The function Autosar Service ID[hex]: 0x01.Synchronous.Non Reentrant function.

Prototype: void Adc_DeInit(void);

3.5.4.2 Function Adc_DisableGroupNotification

Disables the notification mechanism for the requested ADC channel group.

Details:

This function will disable the notification mechanism only for the requested ADC channel group.

Return: void.

Implements: DADC03020.

Note

The function Autosar Service ID[hex]:
0x08.Synchronous.Reentrant function.

Prototype: void Adc_DisableGroupNotification(Adc_GroupType Group);

Table 3-6. Adc_DisableGroupNotification Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.

3.5.4.3 Function Adc_DisableHardwareTrigger

Disables the hardware trigger for the requested ADC Channel group.

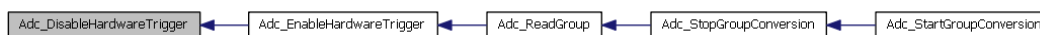


Figure 3-1. Inverse references of fuction Adc_DisableHardwareTrigger

Details:

This function will disable the HW trigger source for the requested ADC channel group.

Return: void.

Implements: DADC03018.

Note

The function Autosar Service ID[hex]:
0x06.Synchronous.Reentrant function.

Prototype: void Adc_DisableHardwareTrigger(Adc_GroupType Group);

Table 3-7. Adc_DisableHardwareTrigger Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.

3.5.4.4 Function Adc_EnableGroupNotification

Enables the notification mechanism for the requested ADC channel group.

Details:

This function will enable the notification mechanism only for the requested ADC channel group.

Return: void.

Implements: DADC03019.

Note

The function Autosar Service ID[hex]:
0x07.Synchronous.Reentrant function.

Prototype: void Adc_EnableGroupNotification(Adc_GroupType Group);

Table 3-8. Adc_EnableGroupNotification Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.

3.5.4.5 Function Adc_EnableHardwareTrigger

Enables the hardware trigger for the requested ADC Channel group.



Figure 3-2. Function Adc_EnableHardwareTrigger References.



Figure 3-3. Inverse references of fuction Adc_EnableHardwareTrigger

Details:

This function will enable the HW trigger source for the requested ADC channel group. This function does set the CTU register for all platform that have the CTU Hw Unit.

Return: void.

Implements: DADC03017.

Note

The function Autosar Service ID[hex]:
0x05.Synchronous.Reentrant function.

Prototype: void Adc_EnableHardwareTrigger(Adc_GroupType Group);

Table 3-9. Adc_EnableHardwareTrigger Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.

3.5.4.6 Function Adc_GetGroupStatus

Returns the conversion status of the requested ADC Channel group.

Details:

This function will return the converison status of the requested ADC channel group.

Return: Adc_StatusType Conversion status for the requested group.

Implements: DADC03021.

Note

The function Autosar Service ID[hex]:
0x09.Synchronous.Reentrant function.

Prototype: `Adc_StatusType Adc_GetGroupStatus(Adc_GroupType Group);`

Table 3-10. Adc_GetGroupStatus Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.

Table 3-11. Adc_GetGroupStatus Return Values

Name	Description
ADC_IDLE	In case of errors.
conversion	Status in case of no errors.

3.5.4.7 Function Adc_GetStreamLastPointer

Returns the number of valid samples per channel.



Figure 3-4. Function Adc_GetStreamLastPointer References.

Details:

Returns the number of valid samples per channel, stored in the result buffer. Reads a pointer, pointing to a position in the group result buffer. With the pointer position, the results of all group channels of the last completed conversion round can be accessed. With the pointer and the return value, all valid group conversion results can be accessed (the user has to take the layout of the result buffer into account).

Return: `Adc_StreamNumSampleType` Number of valid samples per channel.

Implements: DADC03022.

Note

The function Autosar Service ID[hex]:
0x0B.Synchronous.Reentrant function.

Prototype: `Adc_StreamNumSampleType Adc_GetStreamLastPointer(Adc_GroupType Group,
Adc_ValueGroupType **PtrToSamplePtr);`

Table 3-12. Adc_GetStreamLastPointer Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.
Adc_ValueGroupType**	PtrToSamplePtr	output	Pointer to result buffer pointer.

Table 3-13. Adc_GetStreamLastPointer Return Values

Name	Description
0	In case of errors.
>0	Number of valid samples per channel.

3.5.4.8 Function Adc_GetVersionInfo

Returns the version information of this module.

**Figure 3-5. Inverse references of fuction Adc_GetVersionInfo**

Details:

Returns the version information of this module.

Implements: DADC03023.

Note

The function Autosar Service ID[hex]:
0x0A.Synchronous.Reentrant function.

Prototype: void Adc_GetVersionInfo(Std_VersionInfoType *versioninfo);

Table 3-14. Adc_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	versioninfo	output	Pointer to where to store the version information of this module.

Table 3-15. Adc_GetVersionInfo Return Values

Name	Description
structure	In case of no errors.

3.5.4.9 Function Adc_Init

Initializes the ADC hardware unit and the driver.

Details:

This function will initialize both the ADC HW unit and the driver structures.

Return: void.

Post: Initializes the driver.

Implements: DADC03012.

Note

The function Autosar Service ID[hex]: 0x00.Synchronous.Non Reentrant function.

Prototype: void Adc_Init(const Adc_ConfigType *ConfigPtr);

Table 3-16. Adc_Init Arguments

Type	Name	Direction	Description
constAdc_ConfigType*	ConfigPtr	input	Pointer to configuration set in Variant PB (Variant PC requires a NULL_PTR).

3.5.4.10 Function Adc_ReadGroup

Reads the group conversion results.

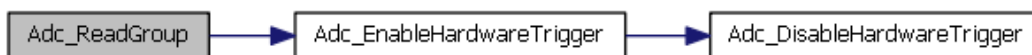


Figure 3-6. Function Adc_ReadGroup References.

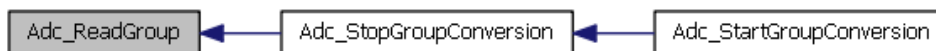


Figure 3-7. Inverse references of fuction Adc_ReadGroup

Details:

Reads the group conversion results of the last completed conversion round of the requested group and stores the channel values starting at the DataBufferPtr address. The group channel values are stored in ascending channel number order (in contrast to the storage layout of the result buffer if streaming access is configured).

Return: Std_ReturnType Standard return type.

Pre: Preconditions as text description. Optional tag.

Post: Postconditions as text description. Optional tag.

Implements: DADC03016.

Note

The function Autosar Service ID[hex]:
0x04.Synchronous.Reentrant function.

Prototype: Std_ReturnType Adc_ReadGroup(Adc_GroupType Group, Adc_ValueGroupType *DataBufferPtr);

Table 3-17. Adc_ReadGroup Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC Channel group.
Adc_ValueGroupType*	DataBufferPtr	input	ADC results of all channels of the selected group are stored in the data buffer addressed with the pointer.

Table 3-18. Adc_ReadGroup Return Values

Name	Description
E_OK	Results are available and written to the data buffer.
E_NOT_OK	No results are available or development error occurred.

3.5.4.11 Function Adc_SetupResultBuffer

Initializes the group specific ADC result buffer pointer as configured to point to the DataBufferPtr address which is passed as parameter.

Details:

Initializes ADC driver with the group specific result buffer start address where the conversion results will be stored. The application has to ensure that the application buffer, where DataBufferPtr points to, can hold all the conversion results of the specified group. The initialization with Adc_SetupResultBuffer is required after reset, before a group conversion can be started.

Return: Std_ReturnType Standard return type.

Pre: ADC422.

Implements: DADC03013.

Note

The function Autosar Service ID[hex]:
0x0C.Synchronous.Reentrant function.

Prototype: Std_ReturnType Adc_SetupResultBuffer(Adc_GroupType Group, Adc_ValueGroupType *DataBufferPtr);

Table 3-19. Adc_SetupResultBuffer Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.
Adc_ValueGroupType*	DataBufferPtr	input	Pointer to result data buffer.

Table 3-20. Adc_SetupResultBuffer Return Values

Name	Description
E_OK	Result buffer pointer initialized correctly.
E_NOT_OK	Operation failed or development error occurred.

3.5.4.12 Function Adc_StartGroupConversion

Starts the conversion of all channels of the requested ADC Channel group.



Figure 3-8. Function Adc_StartGroupConversion References.

Details:

This function will start the SW conversion of all channels of the requested ADC channel group.

Return: void.

Implements: DADC03034.

Note

The function Autosar Service ID[hex]:
0x02.Asynchronous.Reentrant function.

Prototype: void Adc_StartGroupConversion(Adc_GroupType Group);

Table 3-21. Adc_StartGroupConversion Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.

3.5.4.13 Function Adc_StopGroupConversion

Stops the conversion of all channels of the requested ADC Channel group.

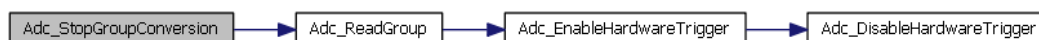


Figure 3-9. Function Adc_StopGroupConversion References.



Figure 3-10. Inverse references of function Adc_StopGroupConversion

Details:

This function will stop the SW conversion of all channels of the requested ADC channel group.

Return: void.

Implements: DADC03015.

Note

The function Autosar Service ID[hex]:
0x03.Synchronous.Reentrant function.

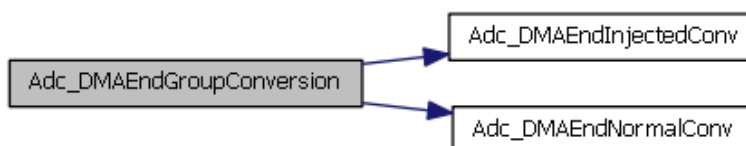
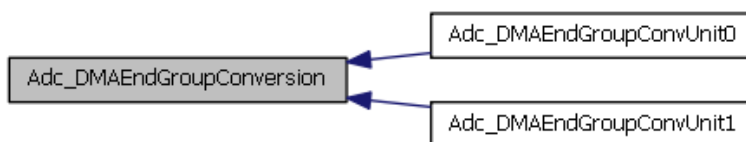
Prototype: void Adc_StopGroupConversion(Adc_GroupType Group);

Table 3-22. Adc_StopGroupConversion Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Numeric ID of requested ADC channel group.

3.5.4.14 Function Adc_DMAEndGroupConversion

This function implements the ISR processing function for the DMA conversions.

**Figure 3-11. Function Adc_DMAEndGroupConversion References.****Figure 3-12. Inverse references of function Adc_DMAEndGroupConversion**

Return: void.

Implements: DADC04015

Prototype: void Adc_DMAEndGroupConversion(Adc_HwUnitType unit);

Table 3-23. Adc_DMAEndGroupConversion Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	The current HW unit.

3.5.4.15 Function Adc_DMAEndInjectedConv

This function handles the Injected conversion via DMA ending.



Figure 3-13. Inverse references of fuction Adc_DMAEndInjectedConv

Details:

When a Injected conversion ends, and the end of conversion DMA ISR is called, this function takes care about the driver and the module state.

Return: void.

Implements: DADC04107

Prototype: void Adc_DMAEndInjectedConv(Adc_HwUnitType unit, Adc_GroupType Group);

Table 3-24. Adc_DMAEndInjectedConv Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	The current HW unit.
Adc_GroupType	Group	input	The handled group.

3.5.4.16 Function Adc_DMAEndNormalConv

This function handles the normal conversion via DMA ending.



Figure 3-14. Inverse references of fuction Adc_DMAEndNormalConv

Details:

When a normal (not injected) conversion ends, and the end of conversion DMA ISR is called, this function takes care about the driver and the module state transitions.

Return: void.

Implements: DADC04011

Prototype: void Adc_DMAEndNormalConv(Adc_HwUnitType unit, Adc_GroupType Group);

Table 3-25. Adc_DMAEndNormalConv Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	The current HW unit.
Adc_GroupType	Group	input	The handled group.

3.5.4.17 Function Adc_EndCTUConv

This function handles the end of a CTU conversion.

**Figure 3-15. Inverse references of fuction Adc_EndCTUConv**

Details:

When a CTU onversion ends, and the end of conversion ISR is called, this function takes care about the driver and the module state transitions.

Return: void.

Implements: DADC04010.

Prototype: void Adc_EndCTUConv(Adc_HwUnitType unit);

Table 3-26. Adc_EndCTUConv Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	The current HW unit.

3.5.4.18 Function Adc_EndGroupConversion

This function implements the ISR for the conversion.

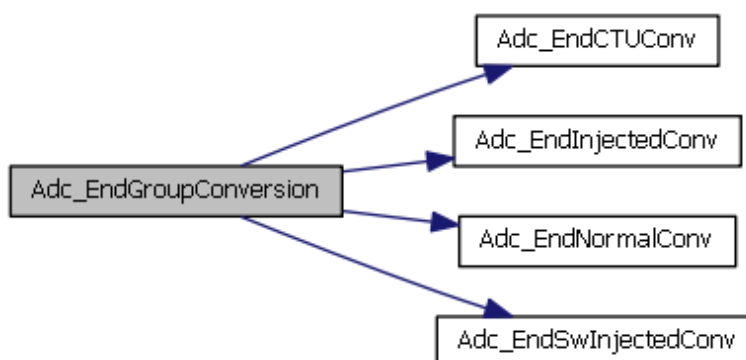


Figure 3-16. Function Adc_EndGroupConversion References.

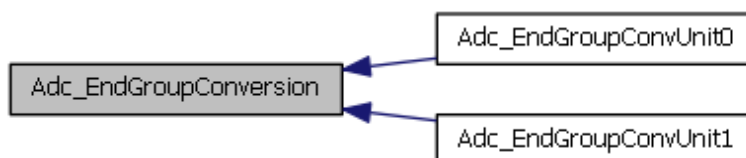


Figure 3-17. Inverse references of function Adc_EndGroupConversion

Details:

The function implements the ISR for the given hardware unit. It's used on both ADC Hardware units.

Return: void.

Implements: DADC04013

Prototype: void Adc_EndGroupConversion(Adc_HwUnitType unit);

Table 3-27. Adc_EndGroupConversion Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	The current HW unit.

3.5.4.19 Function Adc_EndInjectedConv

This function handles the injected conversion ending.



Figure 3-18. Inverse references of fuction Adc_EndInjectedConv

Details:

When a HW injected conversion ends, and the end of conversion ISR is called, this function takes care about the driver and the module state transitions.

Return: void.

Implements: DADC04008.

Prototype: void Adc_EndInjectedConv(Adc_HwUnitType unit);

Table 3-28. Adc_EndInjectedConv Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	The current HW unit.

3.5.4.20 Function Adc_EndNormalConv

This function handles the normal conversion ending.



Figure 3-19. Inverse references of fuction Adc_EndNormalConv

Details:

When a normal (not injected) conversion ends, and the end of conversion ISR is called, this function takes care about the driver and the module state transitions.

Return: void.

Implements: DADC04006.

Prototype: void Adc_EndNormalConv(Adc_HwUnitType unit);

Table 3-29. `Adc_EndNormalConv` Arguments

Type	Name	Direction	Description
<code>Adc_HwUnitType</code>	<code>unit</code>	input	The current HW unit.

3.5.4.21 Function `Adc_EndSwInjectedConv`

This function handles the injected conversion ending.

Figure 3-20. Inverse references of fuction `Adc_EndSwInjectedConv`

Details:

When a SW injected conversion ends, and the end of conversion ISR is called, this function takes care about the driver and the module state transitions.

Return: void.

Implements: DADC04007.

Prototype: `void Adc_EndSwInjectedConv(Adc_HwUnitType unit);`

Table 3-30. `Adc_EndSwInjectedConv` Arguments

Type	Name	Direction	Description
<code>Adc_HwUnitType</code>	<code>unit</code>	input	The current HW unit.

3.5.4.22 Function `Adc_EndWDGInterrupt`

This function handles the watchdog interrupt.

Figure 3-21. Inverse references of fuction `Adc_EndWDGInterrupt`

Details:

This function handles the watchdog interrupt (clears the current channel interrupt, and calls the callback function (if defined)).

Return: void.

Implements: DADC04012

Prototype: void Adc_EndWDGInterrupt(Adc_HwUnitType unit);

Table 3-31. Adc_EndWDGInterrupt Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	The current HW unit.

3.5.4.23 Function Adc_WDGThresholdHandler

This function implements the ISR processing function for the Wdg interrupts.



Figure 3-22. Function Adc_WDGThresholdHandler References.

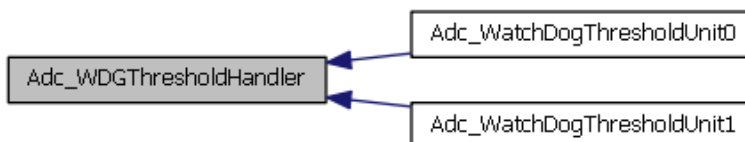


Figure 3-23. Inverse references of fuction Adc_WDGThresholdHandler

Return: void.

Implements: DADC04014

Prototype: void Adc_WDGThresholdHandler(Adc_HwUnitType unit);

Table 3-32. Adc_WDGThresholdHandler Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	The current HW unit.

3.5.4.24 Function Adc_ConfigureThreshold

Function to reconfigure High and Low thresholds for a given group.

Details:

This Non Autosar API is used to reconfigure High and Low thresholds for a given group.

Note

The function Service ID[hex]: 0x14.

Return: Std_ReturnType E_OK or E_NOT_OK.

Prototype: Std_ReturnType Adc_ConfigureThreshold(Adc_GroupType GroupId, Adc_ValueGroupType LowValue, Adc_ValueGroupType HighValue);

Table 3-33. Adc_ConfigureThreshold Arguments

Type	Name	Direction	Description
Adc_GroupType	GroupId	input	Index of group.
Adc_ValueGroupType	LowValue	input	Low threshold value for channels in the group.
Adc_ValueGroupType	HighValue	input	High threshold value for channels in the group.

Table 3-34. Adc_ConfigureThreshold Return Values

Name	Description
E_OK	In case of successfull Configure Threshold.
E_NOT_OK	In case of unsuccessful ConfigureThreshold.

3.5.4.25 Function Adc_DisableChannel

Disable a channel inside a group.

Details:

This function allows to de-active a channel assigned to a group for SW normal conversion

Return: void.

Implements: DADC06013.

Note

The function Service ID[hex]: 0x13.Synchronous.Reentrant
function.Not usable for ANDORRA.

Prototype: void Adc_DisableChannel(Adc_GroupType GroupId, Adc_ChannelType ChannelId);

Table 3-35. Adc_DisableChannel Arguments

Type	Name	Direction	Description
Adc_GroupType	GroupId	input	Symbolic name of group.
Adc_ChannelType	ChannelId	input	Symbolic name of channel.

3.5.4.26 Function Adc_DisableCtuTrigger

Disable the TriggerSource for group selected by Group parameter.

Details:

This non autosar API is used to disable the already enabled TriggerSource of the Group.

Return: void.

Implements: DADC03031.

Note

The function Service ID[hex]: 0x0F.Synchronous.Non
Reentrant function.Not usable for ANDORRA.

Prototype: void Adc_DisableCtuTrigger(Adc_GroupType Group, Adc_HwTriggerTimerType
TriggerSource);

Table 3-36. Adc_DisableCtuTrigger Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Index of group.
Adc_HwTriggerTimerType	TriggerSource	input	Trigger source to be disabled for the group. (Configuration file should contain it for that group).

3.5.4.27 Function Adc_EnableChannel

Enable a channel inside a group.

Details:

This function allows to active a channel assigned to a group for SW normal conversion

Return: void.

Implements: DADC06012.

Note

The function Service ID[hex]: 0x12.Synchronous.Reentrant
function.Not usable for ANDORRA.

Prototype: void Adc_EnableChannel(Adc_GroupType GroupId, Adc_ChannelType ChannelId);

Table 3-37. Adc_EnableChannel Arguments

Type	Name	Direction	Description
Adc_GroupType	GroupId	input	Symbolic name of group.
Adc_ChannelType	ChannelId	input	Symbolic name of channel.

3.5.4.28 Function Adc_EnableCtuTrigger

Enable the TriggerSource for group selected by Group parameter.

Details:

This non autosar API is used to enable any one of the configured TriggerSource of the Group. When this non autosar API is used to enable the trigger source the CTU interrupt will be disabled by the driver. So user has to call the non autosar API Adc_HwResultReadGroup to read the converted result from the ADC hardware register.

Return: void.

Implements: DADC03030

Note

The function Service ID[hex]: 0x0E.Synchronous.Non
Reentrant function.Not usable for ANDORRA.

Prototype: void Adc_EnableCtuTrigger(Adc_GroupType Group, Adc_HwTriggerTimerType TriggerSource);

Table 3-38. Adc_EnableCtuTrigger Arguments

Type	Name	Direction	Description
Adc_GroupType	Group	input	Index of group.
Adc_HwTriggerTimerType	TriggerSource	input	Trigger source to be used for the group. (Configuration file should contain it for that group).

3.5.4.29 Function Adc_GetInjectedConversionStatus

Get the injected conversions status.

Details:

This function checks if an injected conversion (HW/SW) is ongoing

Return: Adc_StatusType Status of the ADC HW unit.

Implements: DADC06014.

Note

The function Service ID[hex]: 0x14.Synchronous.Reentrant function.Not usable for ANDORRA.

Prototype: Adc_StatusType Adc_GetInjectedConversionStatus(Adc_HwUnitType unit);

Table 3-39. Adc_GetInjectedConversionStatus Arguments

Type	Name	Direction	Description
Adc_HwUnitType	unit	input	ADC Unit id.

Table 3-40. Adc_GetInjectedConversionStatus Return Values

Name	Description
ADC_IDLE	SW/HW Injection or Hardware Trigger group are idle.
ADC_BUSY	SW/HW Injection or Hardware Trigger is inprogress.

3.5.4.30 Function `Adc_HwResultReadGroup`

Read the result of the hardware triggered groups conversion result.

Details:

This non autosar API is used to read the result of the hardware triggered groups conversion result from the ADC hardware register in this case the CTU interrupt will be disabled for the group. The VALID bit CDR register will be cleared automatically when we read the results from the channel data register. If the user calls non-autosar function `Adc_HwResultReadGroup()` once again before the next conversion takes place, the `Adc_HwResultReadGroup()` returns `E_NOT_OK`.

Return: `Std_ReturnType` Standard return type.

Implements: DADC03032.

Note

The function Service ID[hex]: 0x0D.Synchronous.Non
Reentrant function.Not usable for ANDORRA.

Prototype: `Std_ReturnType Adc_HwResultReadGroup(Adc_GroupType Group, Adc_ValueGroupType *DataPtr);`

Table 3-41. `Adc_HwResultReadGroup` Arguments

Type	Name	Direction	Description
<code>Adc_GroupType</code>	Group	input	Index of group.
<code>Adc_ValueGroupType*</code>	DataPtr	input	Pointer to a buffer which will be filled by the conversion results.

Table 3-42. `Adc_HwResultReadGroup` Return Values

Name	Description
<code>E_OK</code>	Results are available and written to the data buffer.
<code>E_NOT_OK</code>	No results are available or development error occurred.

3.5.4.31 Function `Adc_SetClockMode`

Set the ADC clock prescaler if available and modify the conversion timings.

Details:

This function set the ADC clock prescaler (Analog clock frequency selector)

Return: Std_ReturnType Standard return type.

Implements: DADC03033.

Note

The function Service ID[hex]: 0x11.Synchronous.Non Reentrant function.Not usable for ANDORRA.

Prototype: Std_ReturnType Adc_SetClockMode(Adc_NonASR_DualClockModeType ClockMode);

Table 3-43. Adc_SetClockMode Arguments

Type	Name	Direction	Description
Adc_NonASR_DualClockModeType	ClockMode	input	Normal or Alternate mode.

Table 3-44. Adc_SetClockMode Return Values

Name	Description
E_OK	In case of successfull settings.
E_NOT_OK	In case of unsuccessful settings.

3.5.4.32 Function Adc_SetMode

Set the ADC mode either to powerdown or normal.

Details:

Set the ADC either to powerdown or normal mode.

Return: Std_ReturnType Standard return type.

Implements: DADC03029

Note

The function Non Autosar Service ID[hex]: 0x10.Synchronous.Non Reentrant function.Not usable for ANDORRA.

Prototype: Std_ReturnType Adc_SetMode(Adc_SetModeType SetMode);

Table 3-45. Adc_SetMode Arguments

Type	Name	Direction	Description
Adc_SetModeType	SetMode	input	Normal or powerdown mode.

Table 3-46. Adc_SetMode Return Values

Name	Description
E_OK	In case of successfull settings.
E_NOT_OK	In case of unsuccessful settings.

3.5.5 Structs Reference

Data structures supported by the ADC driver.

3.5.5.1 Structure Adc_ConfigType

Adc non autosar Enable/Disable invalid group index.

**Figure 3-24. Struct Adc_ConfigType**

Details:

Data structure containing the set of configuration parameters required for initializing the ADC Driver and ADC HW Unit(s).

Implements: DADC06019 Structure for Configuration data.

Implements: DADC04067

Declaration:

```
typedef struct
{
    constAdc_LLD_HWUnitConfiguration* ADC,
    constAdc_LLD_ChannelConfiguration** Channels,
    Adc_GroupType GroupCount,
    constAdc_LLD_GroupConfiguration* Groups,
    constAdc_LLD_MultiConfig Misc
} Adc_ConfigType;
```

Table 3-47. Structure `Adc_ConfigType` member description

Member	Description
ADC	Hw unit configurations.
Channels	Channel configurations.
GroupCount	Total number of groups.
Groups	Group configurations.
Misc	Miscellaneous configuration parameters.

3.5.5.2 Structure `Adc_GroupStatusType`

Structure for group status.

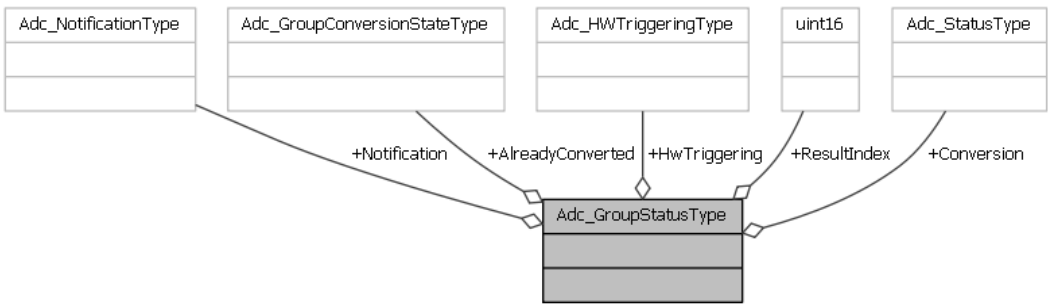


Figure 3-25. Struct `Adc_GroupStatusType`

Details:

This structure contains the group status information.

Implements: DADC03006

Declaration:

```
typedef struct
{
    Adc_GroupConversionStateType AlreadyConverted,
    Adc_StatusType Conversion,
    Adc_HWTriggeringType HwTriggering,
    Adc_NotificationType Notification,
    Adc_StreamNumSampleType ResultIndex
} Adc_GroupStatusType;
```

Table 3-48. Structure `Adc_GroupStatusType` member description

Member	Description
AlreadyConverted	Group was previously converted or not.
Conversion	Group status.

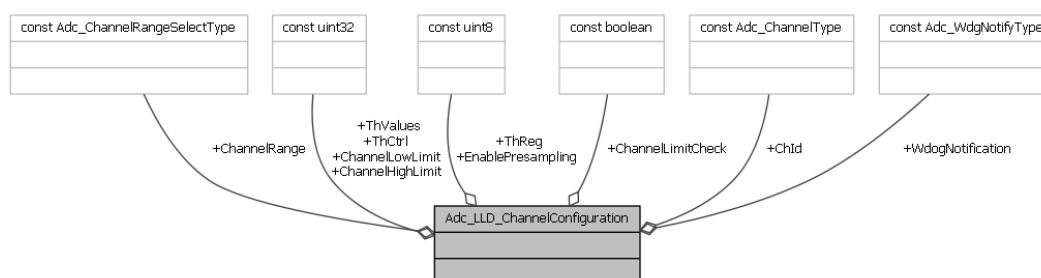
Table continues on the next page...

Table 3-48. Structure Adc_GroupStatusType member description (continued)

Member	Description
HwTriggering	hw trigger enabled/disabled
Notification	notification enabled/disabled
ResultIndex	index into streaming buffer that is currently being filled

3.5.5.3 Structure Adc_LLD_ChannelConfiguration

Structure for channel configuration.

**Figure 3-26. Struct Adc_LLD_ChannelConfiguration**

Implements:

Declaration:

```
typedef struct
{
    const uint32 ChannelHighLimit,
    const boolean ChannelLimitCheck,
    const uint32 ChannelLowLimit,
    const Adc_ChannelRangeSelectType ChannelRange,
    const Adc_ChannelType ChId,
    const uint8 EnablePresampling,
    const uint32 ThCtrl,
    const uint8 ThReg,
    const uint32 ThValues,
    const Adc_WdgNotifyType WdogNotification
} Adc_LLD_ChannelConfiguration;
```

Table 3-49. Structure Adc_LLD_ChannelConfiguration member description

Member	Description
ChannelHighLimit	High limit channel conversion value.
ChannelLimitCheck	Channel limit checking feature.
ChannelLowLimit	Low limit channel conversion value.
ChannelRange	Range conversion.
ChId	Channel Id.

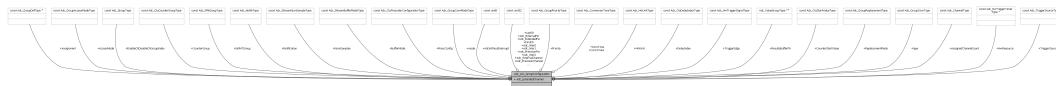
Table continues on the next page...

**Table 3-49. Structure Adc_LLD_ChannelConfiguration member description
(continued)**

Member	Description
EnablePresampling	status to indicate the presampling state
ThCtrl	Threshold register control value.
ThReg	Threshold register configured.
ThValues	Threshold register value.
WdogNotification	Wdg notification function pointer.

3.5.5.4 Structure Adc_LLD_GroupConfiguration

Structure for group configuration.

**Figure 3-27. Struct Adc_LLD_GroupConfiguration**

Implements:

Declaration:

```
typedef struct
{
    constAdc_GroupAccessModeType AccessMode,
    const uint32 Adc_ExtendedChannel,
    const uint32 Adc_ExtendedPsr,
    const uint32 Adc_ExternalChannel,
    const uint32 Adc_ExternalPsr,
    const uint32 Adc_PrecisionChannel,
    const uint32 Adc_PrecisionPsr,
    const uint32 Adc_Wer0,
    const uint32 Adc_Wer1,
    const uint32 Adc_Wer2,
    const uint8 AdcWithoutInterrupt,
    constAdc_ChannelType AssignedChannelCount,
    constAdc_GroupDefType* Assignment,
    constAdc_StreamBufferModeType BufferMode,
    constAdc_ConversionTimeType ConvTime,
    constAdc_ConversionTimeType ConvTime1,
    constAdc_CtuCounterGroupType CounterGroup,
    constAdc_CtuStartValueType CounterStartValue,
    constAdc_CtuDelayIndexType DelayIndex,
    constAdc_GroupType EnableChDisableChGroupIndex,
    const uint32 FirstCh,
    constAdc_HwTriggerTimerType* HwResource,
    constAdc_HwUnitType HWUnit,
    constAdc_MhtGroupType IsMHTGroup,
    const uint32 LastCh,
    constAdc_GroupConvModeType mode,
    constAdc_NotifyType Notification,
    constAdc_StreamNumSampleType NumSamples,
```

```

constAdc_CtuPrescalerConfigurationType PrescConfig,
constAdc_GroupPriorityType Priority,
constAdc_GroupReplacementType ReplcementMode,
Adc_ValueGroupType** ResultsBufferPtr,
constAdc_HwTriggerSignalType TriggerEdge,
constAdc_TriggerSourceType TriggerSource,
constAdc_GroupConvType type
} Adc_LLD_GroupConfiguration;

```

Table 3-50. Structure Adc_LLD_GroupConfiguration member description

Member	Description
AccessMode	Access mode.
Adc_ExtendedChannel	ANS0_31 medium accuracy extended configured channels.
Adc_ExtendedPsr	Presampling for extended channels.
Adc_ExternalChannel	ANX_0_CH_64-ANX_3_CH_95 External configured channels.
Adc_ExternalPsr	Presampling for External channels.
Adc_PrecisionChannel	ANP0_31, Precision configured channels.
Adc_PrecisionPsr	Presampling for Precision channels.
Adc_Wer0	Wer0 for precision channels.
Adc_Wer1	Wer1 for extended(medi.accuracy channels)
Adc_Wer2	Wer2 for external channels.
AdcWithoutInterrupt	Enables or Disables the ADC and DMA interrupts.
AssignedChannelCount	Number of channels.
Assignment	Assigned channels to group.
BufferMode	Buffer Mode.
ConvTime	Conversion time.
ConvTime1	Conversion time CTR1.
CounterGroup	Counter associated with the particular event.
CounterStartValue	Start Value Register for the counter chosen.
DelayIndex	Delay index: Which CTU_SVRx register is used.
EnableChDisableChGroupIndex	Group's index if it has the support to enable/disable channel.
FirstCh	First channel configured.
HwResource	Resource of the selected hw trigger.
HWUnit	Hw unit to which the group belongs to.
IsMHTGroup	Indicate the group type (Regular or MHT)
LastCh	Last channel configured.
mode	Conversion mode (OneShot/Continuous)
Notification	Pointer to notification function.
NumSamples	Number of samples.
PrescConfig	Prescaler configuration.
Priority	Priority of group.
ReplcementMode	Replacement mode.
ResultsBufferPtr	pointer to user result buffer array
TriggerEdge	Hardware trigger edge.
TriggerSource	Hw/Sw trigger.
type	Conversion type (Normal/Injected)

3.5.5.5 Structure Adc_LLD_HWUnitConfiguration

Structure for Adc hardware unit configuration.

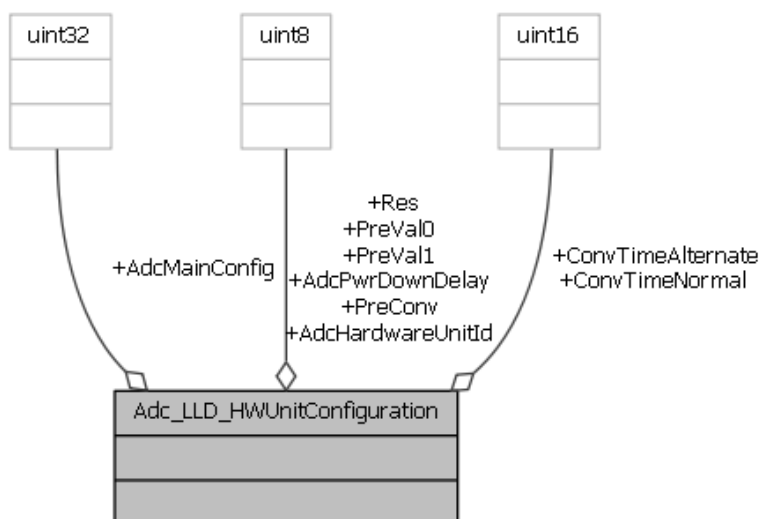


Figure 3-28. Struct Adc_LLD_HWUnitConfiguration

Implements:

Declaration:

```
typedef struct
{
    constAdc_HwUnitType AdcHardwareUnitId,
    const uint32 AdcMainConfig,
    const uint8 AdcPwrDownDelay,
    constAdc_ConversionTimeType ConvTimeAlternate,
    constAdc_ConversionTimeType ConvTimeNormal,
    const uint8 PreConv,
    const uint8 PreVal0,
    const uint8 PreVal1,
    constAdc_ResolutionType Res
} Adc_LLD_HWUnitConfiguration;
```

Table 3-51. Structure Adc_LLD_HWUnitConfiguration member description

Member	Description
AdcHardwareUnitId	Adc hardware unit id.
AdcMainConfig	Main configuration of control registers.
AdcPwrDownDelay	Power down delay for normal power system frequency.
ConvTimeAlternate	
ConvTimeNormal	Normal and Alternate Conversion Timings.
PreConv	presampling control register - preconv field

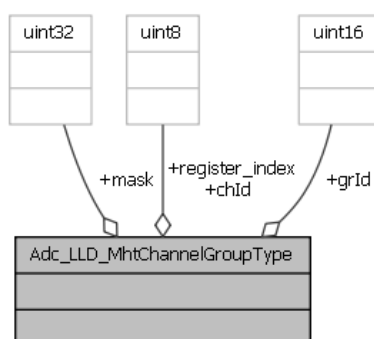
Table continues on the next page...

Table 3-51. Structure Adc_LLD_HWUnitConfiguration member description (continued)

Member	Description
PreVal0	presampling control register - preval0 field
PreVal1	presampling control register - preval1 field
Res	Resolution of the ADC hardware.

3.5.5.6 Structure Adc_LLD_MhtChannelGroupType

This structure is used to manipulate the MHT Multiple Hardware Triggered groups.

**Figure 3-29. Struct Adc_LLD_MhtChannelGroupType**

Implements:

Declaration:

```
typedef struct
{
    Adc_ChannelType chId,
    Adc_GroupType grId,
    uint32 mask,
    uint8 register_index
} Adc_LLD_MhtChannelGroupType;
```

Table 3-52. Structure Adc_LLD_MhtChannelGroupType member description

Member	Description
chId	The ID of a physical ADC channel.
grId	The ID of a MHT group.
mask	Mask value.
register_index	register index value

3.5.5.7 Structure Adc_LLD_MultiConfig

Miscellaneous configuration structure.

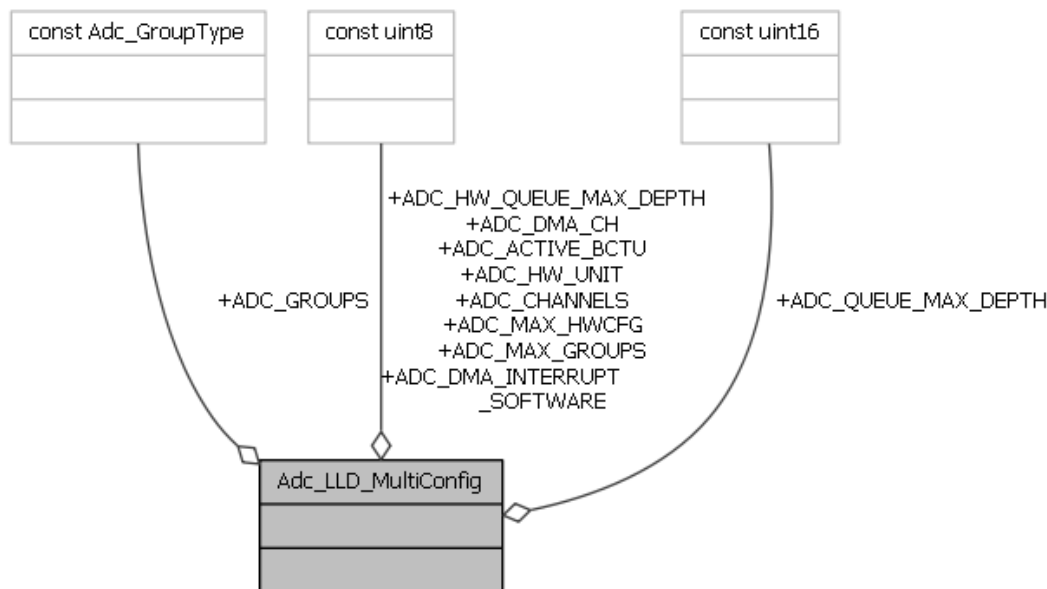


Figure 3-30. Struct Adc_LLD_MultiConfig

Implements:

Declaration:

```
typedef struct
{
    const uint8 ADC_ACTIVE_BCTU,
    const uint8 ADC_CHANNELS[ADC_HW_MAX],
    const uint8 ADC_DMA_CH[ADC_HW_MAX],
    const uint8 ADC_DMA_INTERRUPT_SOFTWARE,
    const Adc_GroupType ADC_GROUPS[ADC_HW_MAX],
    const uint8 ADC_HW_QUEUE_MAX_DEPTH[ADC_HW_MAX],
    const uint8 ADC_HW_UNIT[ADC_HW_MAX],
    const uint8 ADC_MAX_GROUPS,
    const uint8 ADC_MAX_HWCFG,
    const uint16 ADC_QUEUE_MAX_DEPTH
} Adc_LLD_MultiConfig;
```

Table 3-53. Structure Adc_LLD_MultiConfig member description

Member	Description
ADC_ACTIVE_BCTU	Indicate if BCTU controls the ADC.
ADC_CHANNELS	number of groups of the hw unit x
ADC_DMA_CH	If dma driven then indicates the dma channel number for HW UNITS.
ADC_DMA_INTERRUPT_SOFTWARE	DMA or Interrupt driven.

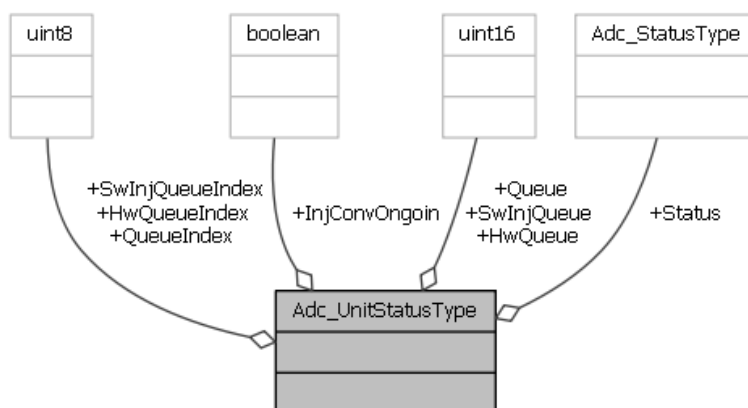
Table continues on the next page...

Table 3-53. Structure Adc_LLD_MultiConfig member description (continued)

Member	Description
ADC_GROUPS	
ADC_HW_QUEUE_MAX_DEPTH	Maximum depth of the hw queue for each unit.
ADC_HW_UNIT	If unit x is active STD_ON/STD_OFF.
ADC_MAX_GROUPS	Number of the maximum groups.
ADC_MAX_HWCFG	Number of the maximum hw units.
ADC_QUEUE_MAX_DEPTH	Depth of the queue.

3.5.5.8 Structure Adc_UnitStatusType

Structure for hardware unit status.

**Figure 3-31. Struct Adc_UnitStatusType**

Details:

This structure contains the HW unit status information.

Implements: DADC03007

Declaration:

```

typedef struct
{
    Adc_GroupType HwQueue[ADC_HW_QUEUE],
        Adc_QueueIndexType HwQueueIndex,
        boolean InjConvOngoin,
        Adc_GroupType Queue[ADC_QUEUE_MAX_DEPTH_MAX],
        Adc_QueueIndexType QueueIndex,
        Adc_StatusType Status,
        Adc_GroupType SwInjQueue[1],
        Adc_QueueIndexType SwInjQueueIndex
} Adc_UnitStatusType;
  
```

Table 3-54. Structure Adc_UnitStatusType member description

Member	Description
HwQueue	The depth of the hardware queue.
HwQueueIndex	Filled slots in the Hw queue.
InjConvOngoin	Indicates Injected conversion or normal conversion.
Queue	Queued groups indexes, always executing Queue[0].
QueueIndex	Filled slots in the queue.
Status	Status of the HW unit: IDLE, BUSY, COMPLETED.
SwInjQueue	The depth of the software injected queue.
SwInjQueueIndex	Filled slots in the Sw injected queue.

3.5.6 Types Reference

Types structures supported by the ADC driver.

3.5.7 Variables Reference

Variables supported by the ADC driver.

3.6 Symbolic Names Disclaimer

All containers having the symbolic name tag set as true in the Autosar schema will generate defines like:

```
#define <Container_Short_Name> <Container_ID>
```

For this reason it is forbidden to duplicate the name of such containers across the MCAL configuration, or to use names that may trigger other compile issues (e.g. match existing #ifdefs arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the ADC Driver. The most of the parameters are described below.

4.1 Configuration elements of Adc

Included forms :

- IMPLEMENTATION_CONFIG_VARIANT
- AdcGeneral
- AdcPublishedInformation
- CommonPublishedInformation
- NonAutosar
- AdcDemEventParameterRefs
- AdcConfigSet

Table 4-1. Revision table

Revision	Date
4.1.0	2010-12-03T17:00:00+01:00

4.2 Form IMPLEMENTATION_CONFIG_VARIANT

Configuration classes. Enable the parameters that are editable for specific configuration classes

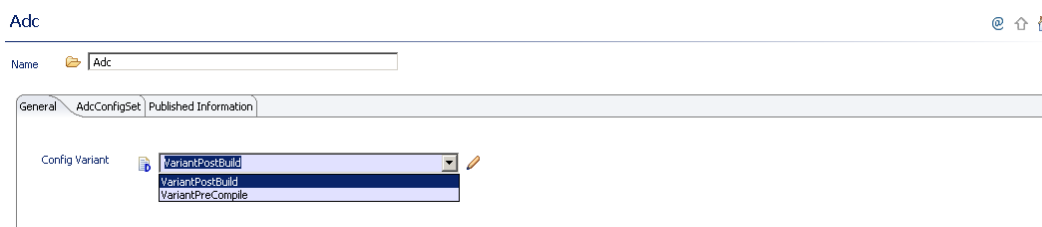


Figure 4-1. Tresos Plugin snapshot for IMPLEMENTATION_CONFIG_VARIANT form.

Table 4-2. Attribute IMPLEMENTATION_CONFIG_VARIANT detailed description

Property	Value
Label	Config Variant
Type	ENUMERATION
Default	VariantPostBuild
Range	VariantPostBuild VariantPreCompile

4.3 Form AdcGeneral

General configuration (parameters) of the ADC Driver software module.

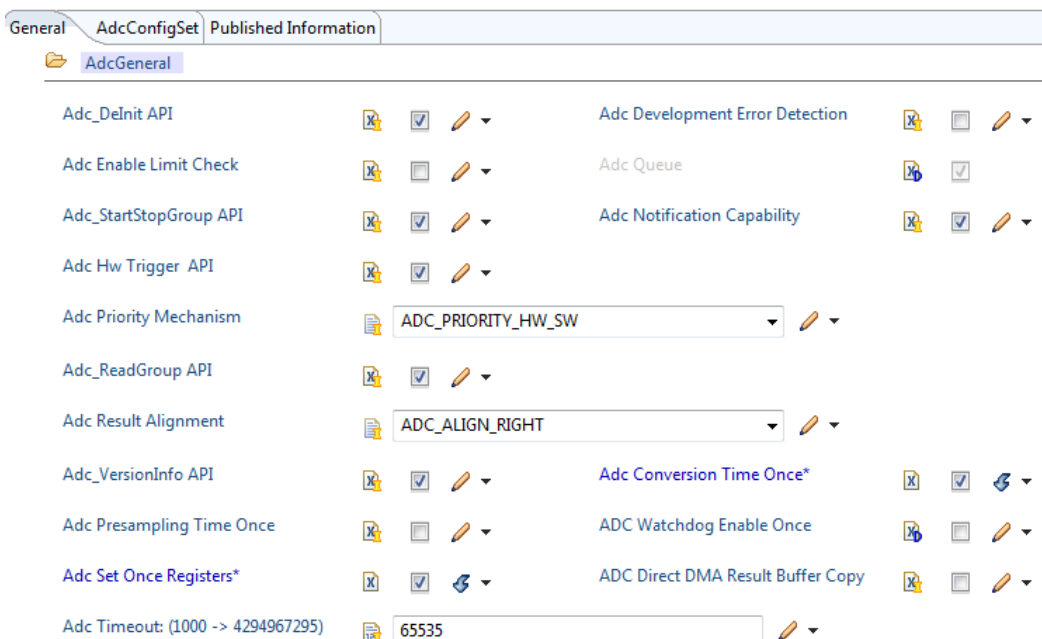


Figure 4-2. Tresos Plugin snapshot for AdcGeneral form.

4.3.1 AdcDeInitApi (AdcGeneral)

Adds/removes the service Adc_DeInit() from the code.

Table 4-3. Attribute AdcDeInitApi (AdcGeneral) detailed description

Property	Value
Label	Adc_DeInit API
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.2 AdcDevErrorDetect (AdcGeneral)

Enable/Disable Development Error Detection.

Table 4-4. Attribute AdcDevErrorDetect (AdcGeneral) detailed description

Property	Value
Label	Adc Development Error Detection
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.3 AdcEnableLimitCheck (AdcGeneral)

Enable/disable limit checking feature in the ADC driver.

Table 4-5. Attribute AdcEnableLimitCheck (AdcGeneral) detailed description

Property	Value
Label	Adc Enable Limit Check
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.4 AdcEnableQueuing (AdcGeneral)

Enable/Disable the Queue. Note that if AdcPriorityImplementation=ADC_PRIORITY_HW_SW this field is always enabled.

Table 4-6. Attribute AdcEnableQueuing (AdcGeneral) detailed description

Property	Value
Label	Adc Queue
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.5 AdcEnableStartStopGroupApi (AdcGeneral)

Adds / removes the services Adc_StartGroupConversion() and Adc_StopGroupConversion from the code.

Table 4-7. Attribute AdcEnableStartStopGroupApi (AdcGeneral) detailed description

Property	Value
Label	Adc_StartStopGroup API
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.6 AdcGrpNotifCapability (AdcGeneral)

Determines, if the group notification mechanism (the functions to enable and disable the notifications) is available at runtime.

Table 4-8. Attribute AdcGrpNotifCapability (AdcGeneral) detailed description

Property	Value
Label	Adc Notification Capability
Type	BOOLEAN

Table continues on the next page...

Table 4-8. Attribute AdcGrpNotifCapability (AdcGeneral) detailed description (continued)

Property	Value
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.7 AdcHwTriggerApi (AdcGeneral)

Adds / removes the services Adc_EnableHardwareTrigger() and Adc_DisableHardwareTrigger() from the code.

Table 4-9. Attribute AdcHwTriggerApi (AdcGeneral) detailed description

Property	Value
Label	Adc Hw Trigger API
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

4.3.8 AdcPriorityImplementation (AdcGeneral)

Select the Priority mechanism. In this version the ADC_PRIORITY_HW isn't used.

Table 4-10. Attribute AdcPriorityImplementation (AdcGeneral) detailed description

Property	Value
Label	Adc Priority Mechanism
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_PRIORITY_NONE
Range	ADC_PRIORITY_HW ADC_PRIORITY_HW_SW ADC_PRIORITY_NONE

4.3.9 AdcReadGroupApi (AdcGeneral)

Adds / removes the service Adc_ReadGroup() from the code.

Table 4-11. Attribute AdcReadGroupApi (AdcGeneral) detailed description

Property	Value
Label	Adc_ReadGroup API
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.10 AdcResultAlignment (AdcGeneral)

Alignment of ADC raw results in ADC result buffer (left/right alignment).

Table 4-12. Attribute AdcResultAlignment (AdcGeneral) detailed description

Property	Value
Label	Adc Result Alignment
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_ALIGN_RIGHT
Range	ADC_ALIGN_LEFT ADC_ALIGN_RIGHT

4.3.11 AdcVersionInfoApi (AdcGeneral)

Adds / removes the service Adc_GetVersionInfo() from the code.

Table 4-13. Attribute AdcVersionInfoApi (AdcGeneral) detailed description

Property	Value
Label	Adc_VersionInfo API
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.12 AdcConvTimeOnce (AdcGeneral)

Implementation Specific Parameter. Enable/Disable one time setting of the registers. If Enabled, the setting of the conversion time registers will be done only once in `Adc_Init()` function for the configured hardware unit.

Table 4-14. Attribute AdcConvTimeOnce (AdcGeneral) detailed description

Property	Value
Label	Adc Conversion Time Once
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.3.13 AdcPreSamplingOnce (AdcGeneral)

Implementation Specific Parameter. Enable/Disable one time setting of the registers. If Enabled, the setting of the presampling time registers will be done only once in `Adc_Init()` function for the configured hardware unit.

Table 4-15. Attribute AdcPreSamplingOnce (AdcGeneral) detailed description

Property	Value
Label	Adc Presampling Time Once
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.3.14 AdcWdgThresholdOnce (AdcGeneral)

Implementation Specific Parameter Enable/Disable one Wdg setting of the registers. If Enabled, the setting of the Watchdog Enable Registers will be done only once in `Adc_Init()` function for the configured hardware unit.

Table 4-16. Attribute AdcWdgThresholdOnce (AdcGeneral) detailed description

Property	Value
Label	ADC Watchdog Enable Once
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.3.15 AdcSetOnceRegisters (AdcGeneral)

Implementation Specific Parameter. Enable/Disable one time setting of the registers. If Enabled, the setting of the presampling time or/and conversion time registers will be done only once in Adc_Init() function for the configured hardware unit.

Table 4-17. Attribute AdcSetOnceRegisters (AdcGeneral) detailed description

Property	Value
Label	Adc Set Once Registers
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.3.16 ADCDirectDMAResultBufferCopy (AdcGeneral)

Enable/Disable the DMA to transfer all channels between the start channel and end channel of the group directly to the user configured buffer without filtering out any unused channels data.

If Enabled, DMA will transfer all the channels between the start channel and end channel even if there are any unused channels inbetween.

If Disabled, DMA will use intermediate buffer to filter out unnecessary channels and copy only used channels data to the user configured buffer.

Note: If the parameter is enabled then the user configured buffer shall accomodate all the channels between the start channel and end channel of the group. The size of user configured buffer shall be (last channel - first channel + 1).

Table 4-18. Attribute ADCDirectDMAResultBufferCopy (AdcGeneral) detailed description

Property	Value
Label	ADC Direct DMA Result Buffer Copy
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.3.17 AdcTimeout (AdcGeneral)

This is a timeout value which is used to wait till - the conversion is not aborted - ADC hardware is not entered in power down state - ADC hardware is not entered in idle state
If the Status is not updated then after this timeout the ADC_E_TIMEOUT production error will be reported and the rest of the functionality will be skipped.

Table 4-19. Attribute AdcTimeout (AdcGeneral) detailed description

Property	Value
Label	Adc Timeout:
Type	INTEGER
Origin	Custom
Symbolic Name	false
Default	65535
Invalid	Range <=4294967295 >=1000

4.3.18 AdcMuxDelaySupport (AdcGeneral)

Note

Enable/disable the External mux delay support in ADC driver.

This is an Implementation Specific Parameter.

Table 4-20. Attribute AdcMuxDelaySupport (AdcGeneral) detailed description

Property	Value
Label	ADC Mux delay Support
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.3.19 AdcMuxDelaySupportPerGroup (AdcGeneral)

Note

Enable/disable the External mux delay write in DSDR during execution of Adc_StartGroupConversion in ADC driver.

This parameter will be enabled only if AdcMuxDelaySupport is true.

This is an Implementation Specific Parameter.

Table 4-21. Attribute AdcMuxDelaySupportPerGroup (AdcGeneral) detailed description

Property	Value
Label	ADC Mux delay Support Per Group
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.4 Form AdcPublishedInformation

Additional published parameters not covered by CommonPublishedInformation container. Note that these parameters do not have any configuration class setting, since they are published information.

The screenshot shows a configuration window titled 'AdcPublishedInformation'. Below the title bar, there is a folder icon and the text 'AdcPublishedInformation'. Three configuration items are listed, each with a lightbulb icon and a text input field:

- Adc Channel Value Signed: false
- Adc Group First Channel Fixed: false
- Adc Max Channel Resolution: 10

Figure 4-3. Tresos Plugin snapshot for AdcPublishedInformation form.

4.4.1 AdcChannelValueSigned (AdcPublishedInformation)

Information whether the result value of the ADC driver has sign information (true) or not (false). If the result shall be interpreted as signed value it shall apply to C-language rules.

Table 4-22. Attribute AdcChannelValueSigned (AdcPublishedInformation) detailed description

Property	Value
Label	Adc Channel Value Signed
Type	BOOLEAN_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

4.4.2 AdcGroupFirstChannelFixed (AdcPublishedInformation)

Information whether the first channel of an ADC Channel group can be configured (false) or is fixed (true) to a value determined by the ADC HW Unit.

Table 4-23. Attribute AdcGroupFirstChannelFixed (AdcPublishedInformation) detailed description

Property	Value
Label	Adc Group First Channel Fixed
Type	BOOLEAN_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

4.4.3 AdcMaxChannelResolution (AdcPublishedInformation)

Maximum Channel resolution in bits (does not specify accuracy).

Table 4-24. Attribute AdcMaxChannelResolution (AdcPublishedInformation) detailed description

Property	Value
Label	Adc Max Channel Resolution
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	10
Invalid	Range <=63 >=1

4.5 Form CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

▼ CommonPublishedInformation

CommonPublishedInformation

ArReleaseMajorVersion	4
ArReleaseMinorVersion	0
ArReleaseRevisionVersion	3
ModuleId	123
SwMajorVersion	1
SwMinorVersion	0
SwPatchVersion	0
VendorApiInfix	
VendorId	43

Figure 4-4. Tresos Plugin snapshot for CommonPublishedInformation form.

4.5.1 ArReleaseMajorVersion (CommonPublishedInformation)

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 4-25. Attribute ArReleaseMajorVersion (CommonPublishedInformation) detailed description

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	4
Invalid	Range >=4 <=4

4.5.2 ArReleaseMinorVersion (CommonPublishedInformation)

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 4-26. Attribute ArReleaseMinorVersion (CommonPublishedInformation) detailed description

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	0
Invalid	Range >=0 <=0

4.5.3 ArReleaseRevisionVersion (CommonPublishedInformation)

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

**Table 4-27. Attribute ArReleaseRevisionVersion
(CommonPublishedInformation) detailed description**

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	3
Invalid	Range >=3 <=3

4.5.4 ModuleId (CommonPublishedInformation)

Module ID of this module from Module List.

Table 4-28. Attribute ModuleId (CommonPublishedInformation) detailed description

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	123
Invalid	Range >=123 <=123

4.5.5 SwMajorVersion (CommonPublishedInformation)

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-29. Attribute SwMajorVersion (CommonPublishedInformation) detailed description

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	1
Invalid	Range

Table 4-29. Attribute SwMajorVersion (CommonPublishedInformation) detailed description

Property	Value
	≥ 1 ≤ 1

4.5.6 SwMinorVersion (CommonPublishedInformation)

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-30. Attribute SwMinorVersion (CommonPublishedInformation) detailed description

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	0
Invalid	Range ≥ 0 ≤ 0

4.5.7 SwPatchVersion (CommonPublishedInformation)

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-31. Attribute SwPatchVersion (CommonPublishedInformation) detailed description

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	1
Invalid	Range ≥ 1 ≤ 1

4.5.8 VendorApiInfix (CommonPublishedInformation)

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name. This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix><Api name from SWS>. E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write. This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Table 4-32. Attribute VendorApiInfix (CommonPublishedInformation) detailed description

Property	Value
Type	STRING_LABEL
Origin	Custom
Symbolic Name	false
Default	
Enable	false

4.5.9 VendorId (CommonPublishedInformation)

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Table 4-33. Attribute VendorId (CommonPublishedInformation) detailed description

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	43
Invalid	Range >=43 <=43

4.6 Form NonAutosar

Non Autosar API settings.

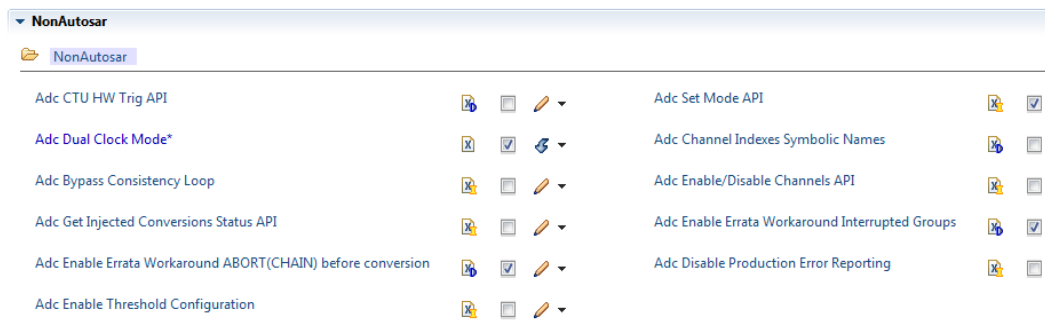


Figure 4-5. Tressos Plugin snapshot for NonAutosar form.

4.6.1 AdcEnableCtuTrigNonAutosarApi (NonAutosar)

This is used to enable the non Autosar API for the hardware triggered group. If this parameter is enabled then `Adc_EnableCtuTrigger()`, `Adc_DisableCtuTrigger()` and `Adc_HwResultReadGroup()` will be available in the driver code. This is an Implementation Specific Parameter. When this parameter is enabled, the result buffer is no longer to be used to read the results as the result will be directly read from HW registers. When this parameter is disabled, normal functionality shall be executed.

Table 4-34. Attribute AdcEnableCtuTrigNonAutosarApi (NonAutosar) detailed description

Property	Value
Label	Adc CTU HW Trig API
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.6.2 AdcSetModeApi (NonAutosar)

Adds/removes the non-autosar implementation api `Adc_SetModeApi()` from the code.

Table 4-35. Attribute AdcSetModeApi (NonAutosar) detailed description

Property	Value
Label	Adc Set Mode API
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	true

4.6.3 AdcEnableDualClockMode (NonAutosar)

Adds/removes the Dual Clock mode service Adc_SetClockMode from the code. Also it enables the programming of Conversion Timing registers in Adc_SetClockMode. This parameter is only available if AdcSetModeApi and AdcConvTimeOnce are both enabled.

Table 4-36. Attribute AdcEnableDualClockMode (NonAutosar) detailed description

Property	Value
Label	Adc Dual Clock Mode
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.6.4 AdcEnableGroupDependentChannelNames (NonAutosar)

This is used to generate ADC symbolic names, that depend also on the ADC group to which each ADC channel is mapped. The generated symbolic name will be something like: #define "ADC_GroupName"_"ADC_ChannelName" "Channel index value", where "Channel index value" is the channel index in the current group. Channel indexes in each group are generated to allow result buffer access by symbolic names.

Table 4-37. Attribute AdcEnableGroupDependentChannelNames (NonAutosar) detailed description

Property	Value
Label	Adc Channel Indexes Symbolic Names
Type	BOOLEAN
Origin	Custom

Table continues on the next page...

Table 4-37. Attribute AdcEnableGroupDependentChannelNames (NonAutosar) detailed description (continued)

Property	Value
Symbolic Name	false
Default	false

4.6.5 AdcBypassConsistencyLoop (NonAutosar)

This is used to increase ADC performances. For instance, when for a group `AdcWithoutInterrupt` is configured, the user does not have to wait the conversion is finished before to abort the current conversion. Nevertheless, as the HW-SW coherency is no longer guaranteed by the driver, the user must make sure he does not call a ADC service before the HW reaches the correct state.

Table 4-38. Attribute AdcBypassConsistencyLoop (NonAutosar) detailed description

Property	Value
Label	Adc Bypass Consistency Loop
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.6.6 AdcEnableChDisableChApi (NonAutosar)

Note

Enable/disable the non-autosar implementation api(s) `Adc_EnableChannel()` and `Adc_DisableChannel()` in ADC driver.

This is an Implementation Specific Parameter.

Table 4-39. Attribute AdcEnableChDisableChApi (NonAutosar) detailed description

Property	Value
Label	Adc Enable/Disable Channels API
Type	BOOLEAN
Origin	Custom

Table continues on the next page...

Table 4-39. Attribute AdcEnableChDisableChApi (NonAutosar) detailed description (continued)

Property	Value
Symbolic Name	false
Default	false

4.6.7 AdcGetInjectedConvStatusApi (NonAutosar)

Note

Enable/disable the non-autosar API Adc_GetInjectedConversionStatus() in ADC driver.

This is an Implementation Specific Parameter.

Table 4-40. Attribute AdcGetInjectedConvStatusApi (NonAutosar) detailed description

Property	Value
Label	Adc Get Injected Conversions Status API
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.6.8 AdcErrataInterruptedNormalGroup (NonAutosar)

Note

If this parameter has been configured to "TRUE", the workaround for the errata, "If CTU chain injected in CTU trigger mode, current normal chain aborted channel might not be restored back", will be enabled.

The Default value is TRUE.

This is an Implementation Specific Parameter.

Table 4-41. Attribute AdcErrataInterruptedNormalGroup (NonAutosar) detailed description

Property	Value
Label	Adc Enable Errata Workaround Interrupted Groups
Type	BOOLEAN

Table continues on the next page...

Table 4-41. Attribute AdcErrataInterruptedNormalGroup (NonAutosar) detailed description (continued)

Property	Value
Origin	Custom
Symbolic Name	false
Default	true

4.6.9 AdcErrataOptimizedSwAbort (NonAutosar)

Note

If this parameter has been configured to "TRUE", the workaround for the errata, "ADC triggering an ABORT or ABORTCHAIN before the conversion starts", will be enabled.

The Default value is TRUE.

This is an Implementation Specific Parameter.

Table 4-42. Attribute AdcErrataOptimizedSwAbort (NonAutosar) detailed description

Property	Value
Label	Adc Enable Errata Workaround ABORT(CHAIN) before conversion
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	true

4.6.10 AdcDisableDemReportErrorStatus (NonAutosar)

AdcDisableDemReportErrorStatus

Switches the Diagnostic Error Reporting and Notification OFF.

Table 4-43. Attribute AdcDisableDemReportErrorStatus (NonAutosar) detailed description

Property	Value
Label	Adc Disable Production Error Reporting
Type	BOOLEAN
Origin	Custom

Table continues on the next page...

Table 4-43. Attribute AdcDisableDemReportErrorStatus (NonAutosar) detailed description (continued)

Property	Value
Symbolic Name	false
Default	false

4.6.11 AdcEnableThresholdConfiguration (NonAutosar)

Note

If this parameter has been configured to "TRUE", the Non-Autosar function "Adc_ConfigureThreshold()" shall be accessible, otherwise this function shall not become generated. Also, there must be at least one channel in the configuration with the threshold feature enabled, otherwise the function will not be generated. This is an Implementation Specific Parameter.

Table 4-44. Attribute AdcEnableThresholdConfiguration (NonAutosar) detailed description

Property	Value
Label	Adc Enable Threshold Configuration
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.7 Form AdcDemEventParameterRefs

Container for the references to DemEventParameter elements which shall be invoked using the API Dem_ReportErrorStatus API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.

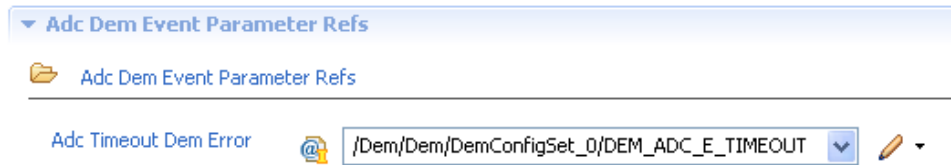


Figure 4-6. Tresos Plugin snapshot for AdcDemEventParameterRefs form.

4.7.1 ADC_E_TIMEOUT (AdcDemEventParameterRefs)

Reference to configured DEM event to report "Timeout failure".

Table 4-45. Attribute ADC_E_TIMEOUT (AdcDemEventParameterRefs) detailed description

Property	Value
Label	Adc Timeout Dem Error
Type	SYMBOLIC-NAME-REFERENCE
Origin	Custom
Enable	true

4.8 Form AdcConfigSet

This is the base container that contains the post-build selectable configuration parameters

Included forms :

- [Form AdcGeneric](#)
- [Form AdcHwUnit](#)

AdcConfigSet

Name*

General AdcHwUnit

AdcGeneric

AdcGeneric

Adc Max Queue Depth (1 -> 255)*

Adc Hw unit 0 Max Group Channels (0 -> 4294967295)*

Adc Hw unit 1 Max Group Channels (0 -> 4294967295)*

Adc Transfer Type*

Figure 4-7. TRESOS Plugin snapshot for AdcConfigSet form.

4.8.1 Form AdcGeneric

General configuration parameters for the current configuration of the ADC Driver software module.

Is included by form : [Form AdcConfigSet](#)

AdcGeneric

AdcGeneric

Adc Max Queue Depth (1 -> 255)*

Adc Hw unit 0 Max Group Channels (0 -> 4294967295)*

Adc Hw unit 1 Max Group Channels (0 -> 4294967295)*

Adc Transfer Type*

Figure 4-8. TRESOS Plugin snapshot for AdcGeneric form.

4.8.1.1 AdcPriorityQueueMaxDepth (AdcGeneric)

Maximum depth of queue used for queuing of incoming conversion requests when hardware unit is busy.

Table 4-46. Attribute AdcPriorityQueueMaxDepth (AdcGeneric) detailed description

Property	Value
Label	Adc Max Queue Depth
Type	INTEGER
Origin	Custom
Symbolic Name	false
Default	1
Invalid	Range <=255 >=1

4.8.1.2 AdcTransferType (AdcGeneric)

Select the Interrupt or Dma transfer Type. If DMA is used, user must not run SW and HW groups at the same time on the same HW unit because the same DMA channel will be used for both. If DMA is required for AdcTransferType, it is recommended to keep AdcWithoutInterrupts as false, otherwise, DMA will not be configured and it will be user responsibility to read the results from registers directly by calling Adc_ReadGroup.

Table 4-47. Attribute AdcTransferType (AdcGeneric) detailed description

Property	Value
Label	Adc Transfer Type
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_INTERRUPT
Range	ADC_DMA ADC_INTERRUPT

4.8.2 Form AdcHwUnit

This container contains the Driver configuration (parameters) depending on grouping of channels.

Is included by form : [Form AdcConfigSet](#)

Included forms :

- [Form AdcNormalAltConvTimings](#)

- [Form AdcChannel](#)
- [Form AdcGroup](#)

The screenshot shows the 'AdcHwUnit' configuration window. At the top, the 'Name' field is set to 'AdcHwUnit_0'. Below this are three tabs: 'General', 'AdcChannel', and 'AdcGroup'. The 'General' tab is selected. It contains several configuration fields: 'Adc Source Clock*' is a dropdown menu set to 'CLK_SRC_0'; 'Adc Hardware Unit*' is a dropdown menu set to 'ADC_UNIT_0'; 'Adc Prescaler Value (0 -> 65535)*' is a text input field set to '2'; 'Adc Power Down Delay (0 -> 255)*' is a text input field set to '15'; 'ADC Mux Delay (0 -> 4095)*' is a text input field set to '15'; 'Adc Auto Clock Off*' is a checkbox that is unchecked; 'Bypass Sampling*' is a checkbox that is unchecked; and 'Adc Presampling channel enable signal*' is a dropdown menu set to 'ADC_PREAMP_VOLTAGE_V0'.

Figure 4-9. Tresos Plugin snapshot for AdcHwUnit form.

4.8.2.1 AdcClockSource (AdcHwUnit)

Not used. This value should be selected in the MCU Driver.

Table 4-48. Attribute AdcClockSource (AdcHwUnit) detailed description

Property	Value
Label	Adc Source Clock
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	CLK_SRC_0
Enable	false
Range	CLK_SRC_0

4.8.2.2 AdcHwUnitId (AdcHwUnit)

Specifies the used ADC Hardware Unit (for Bolero 128k,256k,512k - only ADC0; for Bolero 1M, 1.5M - ADC0 and ADC1 are available).

Table 4-49. Attribute AdcHwUnitId (AdcHwUnit) detailed description

Property	Value
Label	Adc Hardware Unit
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Enable	true
Range	ADC_UNIT_0 ADC_UNIT_1

4.8.2.3 AdcPrescale (AdcHwUnit)

The prescaler Value is fixed to two.

Table 4-50. Attribute AdcPrescale (AdcHwUnit) detailed description

Property	Value
Label	Adc Prescaler Value
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	2
Enable	true
Invalid	Range ≤65535 ≥0

4.8.2.4 AdcPowerDownDelay (AdcHwUnit)

The delay between the power down bit reset and the starting of conversion when ADC runs on normal system frequency.

Table 4-51. Attribute AdcPowerDownDelay (AdcHwUnit) detailed description

Property	Value
Label	Adc Power Down Delay
Type	INTEGER
Origin	Custom
Symbolic Name	false

Table continues on the next page...

Table 4-51. Attribute AdcPowerDownDelay (AdcHwUnit) detailed description (continued)

Property	Value
Default	15
Invalid	Range >=0 <=255

4.8.2.5 AdcAltPowerDownDelay (AdcHwUnit)

The delay between the power down bit reset and the starting of conversion when ADC runs on low power system frequency.

Table 4-52. Attribute AdcAltPowerDownDelay (AdcHwUnit) detailed description

Property	Value
Label	Adc Alternate Power Down Delay
Type	INTEGER
Origin	Custom
Symbolic Name	false
Default	15
Invalid	Range >=0 <=255

4.8.2.6 AdcMuxDelay (AdcHwUnit)

The delay between the external decode signals and the start of the sampling phase.

It is used to take into account the settling time of the external mux when ADC runs on normal system frequency.

The decode signal delay is calculated as (DSD X 1/frequency of system clock).

The DSDR register is 12-bit.

Note: This is an Implementation Specific Parameter.

Table 4-53. Attribute AdcMuxDelay (AdcHwUnit) detailed description

Property	Value
Label	ADC Mux Delay

Table continues on the next page...

Table 4-53. Attribute AdcMuxDelay (AdcHwUnit) detailed description (continued)

Property	Value
Type	INTEGER
Origin	Custom
Symbolic Name	false
Default	15
Invalid	Range >=0 <=4095

4.8.2.7 AdcAutoClockOff (AdcHwUnit)

Enables/disables the auto-clock-off features.

Table 4-54. Attribute AdcAutoClockOff (AdcHwUnit) detailed description

Property	Value
Label	Adc Auto Clock Off
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.8.2.8 AdcBypassSampling (AdcHwUnit)

When true, this parameter bypasses the sampling phase for all the presampling enabled channels, for the current hardware unit. The normal operation sequence on the presampling enabled channels will be: Presampling -> Conversion.

Table 4-55. Attribute AdcBypassSampling (AdcHwUnit) detailed description

Property	Value
Label	Bypass Sampling
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false
Enable	true

4.8.2.9 AdcPresamplingEnableSignal (AdcHwUnit)

Select the presampling channel enable signal for the current hardware unit:

ADC_PRESAMP_VOLTAGE_V0 - $V_{ss} = 0V$ ADC_PRESAMP_VOLTAGE_V1 - $V_{dd} = 5V$.

Table 4-56. Attribute AdcPresamplingEnableSignal (AdcHwUnit) detailed description

Property	Value
Label	Adc Presampling channel enable signal
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_PRESAMP_VOLTAGE_V0
Range	ADC_PRESAMP_VOLTAGE_V0 ADC_PRESAMP_VOLTAGE_V1

4.8.2.10 Form AdcNormalAltConvTimings

Selects Normal and Alternate values used in Adc_SetClockMode API for programming CTR Conversion Timing Registers. This is available only when AdcEnableDualClockMode has been enabled

Is included by form : [Form AdcHwUnit](#)

AdcHwUnit @ ↑

Name AdcHwUnit_0_0

General **AdcChannel** AdcGroup

▼ **AdcNormalAltConvTimings**

AdcNormalAltConvTimings

Normal Latch Duration*	ADC_ONE_CLOCK_CYCLE	
Normal Comparison Duration*	ADC_THREE_LATCH_CYCLE	
Normal Sampling Duration for Channels 0..14 (3 -> 255)*	16	
Alternate Latch Duration	ADC_HALF_CLOCK_CYCLE	
Alternate Comparison Duration	ADC_TWO_LATCH_CYCLE	
Alternate Sampling Duration for Channels 0..14 (3 -> 255)	8	

Figure 4-10. TRESOS Plugin snapshot for AdcNormalAltConvTimings form.

4.8.2.10.1 AdcLatchingDurationNormal (AdcNormalAltConvTimings)

Select the Normal Bit Latching Phase Duration used when calling `Adc_SetClockMode(ADC_NORMAL)`. Usually this value should be `ADC_ONE_CLOCK_CYCLE`.

Table 4-57. Attribute AdcLatchingDurationNormal (AdcNormalAltConvTimings) detailed description

Property	Value
Label	Normal Latch Duration
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_HALF_CLOCK_CYCLE
Range	ADC_HALF_CLOCK_CYCLE ADC_ONE_CLOCK_CYCLE

4.8.2.10.2 AdcComparisonDurationNormal (AdcNormalAltConvTimings)

Select the Normal Comparison Duration used when calling `Adc_SetClockMode(ADC_NORMAL)`. Usually this value should be `ADC_THREE_LATCH_CYCLE`.

Table 4-58. Attribute AdcComparisonDurationNormal (AdcNormalAltConvTimings) detailed description

Property	Value
Label	Normal Comparison Duration
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_TWO_LATCH_CYCLE
Range	ADC_ONE_LATCH_CYCLE ADC_TWO_LATCH_CYCLE ADC_THREE_LATCH_CYCLE ADC_FOUR_LATCH_CYCLE

4.8.2.10.3 AdcSamplingDurationNormal (AdcNormalAltConvTimings)

Selects the Normal INPSAMP value of the CTR register used when calling `Adc_SetClockMode(ADC_NORMAL)`

Table 4-59. Attribute AdcSamplingDurationNormal (AdcNormalAltConvTimings) detailed description

Property	Value
Label	Normal Sampling Duration for Channels 0..14
Type	INTEGER
Origin	Custom
Symbolic Name	false
Invalid	Range >=3 <=255

4.8.2.10.4 AdcLatchingDurationAlt (AdcNormalAltConvTimings)

Select the Alternate Bit Latching Phase Duration used when calling `Adc_SetClockMode(ADC_ALTERNATE)`. Usually this value should be `ADC_ONE_CLOCK_CYCLE`.

Table 4-60. Attribute AdcLatchingDurationAlt (AdcNormalAltConvTimings) detailed description

Property	Value
Label	Alternate Latch Duration
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_HALF_CLOCK_CYCLE
Range	ADC_HALF_CLOCK_CYCLE ADC_ONE_CLOCK_CYCLE

4.8.2.10.5 AdcComparisonDurationAlt (AdcNormalAltConvTimings)

Select the Alternate Comparison Duration used when calling `Adc_SetClockMode(ADC_ALTERNATE)`. Usually this value should be `ADC_THREE_LATCH_CYCLE`.

Table 4-61. Attribute AdcComparisonDurationAlt (AdcNormalAltConvTimings) detailed description

Property	Value
Label	Alternate Comparison Duration
Type	ENUMERATION
Origin	Custom

Table continues on the next page...

Table 4-61. Attribute AdcComparisonDurationAlt (AdcNormalAltConvTimings) detailed description (continued)

Property	Value
Symbolic Name	false
Default	ADC_TWO_LATCH_CYCLE
Range	ADC_ONE_LATCH_CYCLE ADC_TWO_LATCH_CYCLE ADC_THREE_LATCH_CYCLE ADC_FOUR_LATCH_CYCLE

4.8.2.10.6 AdcSamplingDurationAlt (AdcNormalAltConvTimings)

Selects the Normal INPSAMP value of the CTR register used when calling `Adc_SetClockMode(ADC_ALTERNATE)`

Table 4-62. Attribute AdcSamplingDurationAlt (AdcNormalAltConvTimings) detailed description

Property	Value
Label	Alternate Sampling Duration for Channels 0..14
Type	INTEGER
Origin	Custom
Symbolic Name	false
Invalid	Range >=3 <=255

4.8.2.11 Form AdcChannel

This container contains the channel configuration (parameters) depending on the hardware capability.

Is included by form : [Form AdcHwUnit](#)

Included forms :

- [Form AdcThresholdControl](#)

AdcChannel

Name* AdcChannel_0

General

Adc Channel Conversion Time (0 -> 9223372036854775807)*

0

Adc Channel High Limit (0 -> 9223372036854775807)*

0

Adc Channel ID (0 -> 1024)*

0

Adc Physical HW Channel*

Adc Channel Limit Check*

☐

Adc Channel Low Limit (0 -> 9223372036854775807)*

0

Adc Channel Range Select*

ADC_RANGE_ALWAYS

Adc High Reference Voltage*

UPPER_REF_VOLT_0

Adc Low Reference Voltage*

LOWER_REF_VOLT_0

Adc Channel Resolution (1 -> 63)*

12

Adc Channel Sampling time (7 -> 254)*

7

Adc Enable Presampling*

☐

AdcThresholdControl

AdcThresholdControl

Adc Enable Threshold*

☐

Adc Threshold Register*

ADC_THRESHOLD_REG_0

Adc High Threshold value (0 -> 4095)*

4095

Adc Low Threshold value (0 -> 4095)*

0

Adc Channel Watchdog Notification*

NULL_PTR

Figure 4-11. Tresos Plugin snapshot for AdcChannel form.

4.8.2.11.1 AdcChannelConvTime (AdcChannel)

Configuration of conversion time, i.e. the time during which the analogue value is converted into digital representation, (in clock cycles) for each channel, if supported by hardware. This parameter is not used by the current implementation.

Table 4-63. Attribute AdcChannelConvTime (AdcChannel) detailed description

Property	Value
Label	Adc Channel Conversion Time
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Enable	false
Invalid	Range <=9223372036854775807 >=0

4.8.2.11.2 AdcChannelHighLimit (AdcChannel)

High limit - used for limit checking.

If configured alignment of ADC raw results in ADC result buffer is left alignment than during generation time the value of AdcChannelhighlimit will be shifted left by $(15 - \text{Adc_resolution} + 1)$ times.

Table 4-64. Attribute AdcChannelHighLimit (AdcChannel) detailed description

Property	Value
Label	Adc Channel High Limit
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range ≤ 9223372036854775807 ≥ 0

4.8.2.11.3 AdcChannelId (AdcChannel)

This parameter defines the assignment of the channel to the physical ADC hardware channel.

Table 4-65. Attribute AdcChannelId (AdcChannel) detailed description

Property	Value
Label	Adc Channel ID
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Invalid	Range ≤ 1024 ≥ 0

4.8.2.11.4 AdcHwChannel (AdcChannel)

Selects the physical Hardware Adc Channel. Note: Range of the ADC Channels depends on the selected package.

Table 4-66. Attribute AdcHwChannel (AdcChannel) detailed description

Property	Value
Label	Adc Physical HW Channel
Type	ENUMERATION
Origin	Custom
Symbolic Name	false

4.8.2.11.5 AdcChannelLimitCheck (AdcChannel)

Enables or disables limit checking for an ADC channel.

Table 4-67. Attribute AdcChannelLimitCheck (AdcChannel) detailed description

Property	Value
Label	Adc Channel Limit Check
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

4.8.2.11.6 AdcChannelLowLimit (AdcChannel)

Low limit - used for limit checking.

If configured alignment of ADC raw results in ADC result buffer is left alignment than during generation time the value of AdcChannelLowlimit will be shifted left by $(15 - \text{Adc_resolution} + 1)$ times.

Table 4-68. Attribute AdcChannelLowLimit (AdcChannel) detailed description

Property	Value
Label	Adc Channel Low Limit
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range ≤ 9223372036854775807 ≥ 0

4.8.2.11.7 AdcChannelRangeSelect (AdcChannel)

In case of active limit checking: defines which conversion values are taken into account related to the borders defined with AdcChannelLowLimit and AdcChannelHighLimit.

Table 4-69. Attribute AdcChannelRangeSelect (AdcChannel) detailed description

Property	Value
Label	Adc Channel Range Select
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_RANGE_ALWAYS
Range	ADC_RANGE_ALWAYS ADC_RANGE_BETWEEN ADC_RANGE_NOT_BETWEEN ADC_RANGE_NOT_OVER_HIGH ADC_RANGE_NOT_UNDER_LOW ADC_RANGE_OVER_HIGH ADC_RANGE_UNDER_LOW

4.8.2.11.8 AdcChannelRefVoltsrcHigh (AdcChannel)

Upper reference voltage source for each channel. This parameter is not used by the current implementation.

Table 4-70. Attribute AdcChannelRefVoltsrcHigh (AdcChannel) detailed description

Property	Value
Label	Adc High Reference Voltage
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	UPPER_REF_VOLT_0
Enable	false
Range	UPPER_REF_VOLT_0

4.8.2.11.9 AdcChannelRefVoltsrcLow (AdcChannel)

Lower reference voltage source for each channel. This parameter is not used by the current implementation.

Table 4-71. Attribute AdcChannelRefVoltsrcLow (AdcChannel) detailed description

Property	Value
Label	Adc Low Reference Voltage
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	LOWER_REF_VOLT_0
Enable	false
Range	LOWER_REF_VOLT_0

4.8.2.11.10 AdcChannelResolution (AdcChannel)**Table 4-72. Detailed description.**

mpc560[1,2]d, spc560d[30,40]	12	NA
mpc560[2,3,4][b,c], spc560[b,c] [40,44,50]	10	NA
mpc560[5,6,7]b, spc560b[54,60,64]	10	12

Table 4-73. Attribute AdcChannelResolution (AdcChannel) detailed description

Property	Value
Label	Adc Channel Resolution
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Enable	true
Invalid	Range <=63 >=1

4.8.2.11.11 AdcChannelSampTime (AdcChannel)

Sampling time, i.e. the time during which the value is sampled, (in clock cycles) for each channel. Not used.

Table 4-74. Attribute AdcChannelSampTime (AdcChannel) detailed description

Property	Value
Label	Adc Channel Sampling time
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	7
Enable	false
Invalid	Range >=7 <=254

4.8.2.11.12 AdcEnablePresampling (AdcChannel)

When true, this parameter enables the presampling phase for the selected channel. The normal operation sequence on the channel: Presampling -> Sampling -> Conversion.

Table 4-75. Attribute AdcEnablePresampling (AdcChannel) detailed description

Property	Value
Label	Adc Enable Presampling
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.8.2.11.13 Form AdcThresholdControl

This container contains the Threshold configuration (parameters).

Is included by form : [Form AdcChannel](#)

Figure 4-12. TresoS Plugin snapshot for AdcThresholdControl form.

4.8.2.11.13.1 AdcEnableThresholds (AdcThresholdControl)

When true, this parameter enables the threshold detection feature for the selected channel.

Table 4-76. Attribute AdcEnableThresholds (AdcThresholdControl) detailed description

Property	Value
Label	Adc Enable Threshold
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.8.2.11.13.2 AdcThresholdControlRegister (AdcThresholdControl)

Select the threshold register which provides the values to be used for upper and lower thresholds.

For 5601/2D: ADCHwUnit0(12Bit ADC) supports threshold registers from ADC_THRESHOLD_REG_0 to ADC_THRESHOLD_REG_2.

For 5602/3/4B: ADCHwUnit0(10Bit ADC) supports threshold registers from ADC_THRESHOLD_REG_0 to ADC_THRESHOLD_REG_3.

For 5605/6/7B: ADCHwUnit0(10Bit ADC) supports threshold registers from ADC_THRESHOLD_REG_0 to ADC_THRESHOLD_REG_5 while ADCHwUnit1(12Bit ADC) supports threshold registers from ADC_THRESHOLD_REG_0 to ADC_THRESHOLD_REG_2.

Note: This is an Implementation Specific Parameter.

Table 4-77. Attribute AdcThresholdControlRegister (AdcThresholdControl) detailed description

Property	Value
Label	Adc Threshold Register
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_THRESHOLD_REG_0
Range	ADC_THRESHOLD_REG_0 ADC_THRESHOLD_REG_1 ADC_THRESHOLD_REG_2 ADC_THRESHOLD_REG_3 ADC_THRESHOLD_REG_4 ADC_THRESHOLD_REG_5

4.8.2.11.13.3 AdcHighThreshold (AdcThresholdControl)

Set the value for High Threshold.

Table 4-78. Attribute AdcHighThreshold (AdcThresholdControl) detailed description

Property	Value
Label	Adc High Threshold value
Type	INTEGER
Origin	Custom
Symbolic Name	false
Default	1023
Invalid	Range >=0 <=4095

4.8.2.11.13.4 AdcLowThreshold (AdcThresholdControl)

Set the value for Low Threshold.

Table 4-79. Attribute AdcLowThreshold (AdcThresholdControl) detailed description

Property	Value
Label	Adc Low Threshold value
Type	INTEGER
Origin	Custom

Table continues on the next page...

Table 4-79. Attribute AdcLowThreshold (AdcThresholdControl) detailed description (continued)

Property	Value
Symbolic Name	false
Default	0
Invalid	Range >=0 <=4095

4.8.2.11.13.5 AdcWdogNotification (AdcThresholdControl)

This function pointer is called everytime when the conversion of the channel caused a watchdog interrupt.

Table 4-80. Attribute AdcWdogNotification (AdcThresholdControl) detailed description

Property	Value
Label	Adc Channel Watchdog Notification
Type	FUNCTION-NAME
Origin	Custom
Symbolic Name	false
Default	NULL_PTR

4.8.2.12 Form AdcGroup

This container contains the Group configuration (parameters).

Is included by form : [Form AdcHwUnit](#)

Included forms :

- [Form AdcGroupDefinition](#)
- [Form AdcGroupConversionConfiguration](#)
- [Form AdcHwTrig](#)

The screenshot shows the 'AdcGroup' configuration window. At the top, the 'Name' field is set to 'AdcGroup_0'. The 'General' tab is selected, showing the following parameters:

- Adc Group Access Mode*: ADC_ACCESS_MODE_SINGLE
- Adc Group Conversion Mode*: ADC_CONV_MODE_ONESHOT
- Adc Group Conversion Type*: ADC_CONV_TYPE_NORMAL
- Adc Group Id (0 -> 1023)*: 0
- Adc Group Priority (0 -> 255)*: 0
- Adc Group Replacement*: ADC_GROUP_REPL_ABORT_RESTART
- Adc Group Trigger Source*: ADC_TRIGG_SRC_SW
- Adc Group Trigger Signal*: ADC_HW_TRIG_RISING_EDGE
- Adc Group Trigger Timer (0 -> 9223372036854775807)*: 0
- Adc Group Notification*: NULL_PTR
- Adc Group Streaming Buffer Mode*: ADC_STREAM_BUFFER_LINEAR
- Adc Group Streaming Number Samples (1 -> 255)*: 1
- Adc Group Enable/Disable channels*: ☐ (Adc Group Without Interrupts* is also present with a checkbox)
- Adc MHT Group*: ☐

Below the main configuration, there is a section for 'AdcGroupConversionConfiguration' with the following parameters:

- Adc Group Latch Duration*: ADC_HALF_CLOCK_CYCLE
- Adc Group Comparison Duration*: ADC_TWO_LATCH_CYCLE

Figure 4-13. TresoS Plugin snapshot for AdcGroup form.

4.8.2.12.1 AdcGroupAccessMode (AdcGroup)

Type of access mode to group conversion results.

Table 4-81. Attribute AdcGroupAccessMode (AdcGroup) detailed description

Property	Value
Label	Adc Group Access Mode
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_ACCESS_MODE_SINGLE
Range	ADC_ACCESS_MODE_SINGLE ADC_ACCESS_MODE_STREAMING

4.8.2.12.2 AdcGroupConversionMode (AdcGroup)

Type of Conversion mode of the channel group.

Table 4-82. Attribute AdcGroupConversionMode (AdcGroup) detailed description

Property	Value
Label	Adc Group Conversion Mode
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_CONV_MODE_ONESHOT
Range	ADC_CONV_MODE_CONTINUOUS ADC_CONV_MODE_ONESHOT

4.8.2.12.3 AdcGroupConversionType (AdcGroup)

Normal or Injected conversion type.

Table 4-83. Attribute AdcGroupConversionType (AdcGroup) detailed description

Property	Value
Label	Adc Group Conversion Type
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_CONV_TYPE_NORMAL
Range	ADC_CONV_TYPE_NORMAL ADC_CONV_TYPE_INJECTED

4.8.2.12.4 AdcGroupId (AdcGroup)

Numeric ID of the group. This parameter is the symbolic name to be used on the API. This symbolic name allows accessing Channel Group data. This value will be assigned to the symbolic name derived of the AdcGroup container shortName.

Table 4-84. Attribute AdcGroupId (AdcGroup) detailed description

Property	Value
Label	Adc Group Id
Type	INTEGER

Table continues on the next page...

Table 4-84. Attribute AdcGroupId (AdcGroup) detailed description (continued)

Property	Value
Origin	AUTOSAR_ECUC
Symbolic Name	true
Invalid	Range <=1023 >=0

4.8.2.12.5 AdcGroupPriority (AdcGroup)

Priority level of the AdcGroup. This item is ignored if Adc/AdcGeneral/AdcPriorityImplementation is defined to ADC_PRIORITY_NONE.

Table 4-85. Attribute AdcGroupPriority (AdcGroup) detailed description

Property	Value
Label	Adc Group Priority
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Enable	true
Invalid	Range <=255 >=0

4.8.2.12.6 AdcGroupReplacement (AdcGroup)

Replacement mechanism used on ADC group level, if a group conversion is interrupted by a group which has a higher priority. It's fixed to Abort/Restart

Table 4-86. Attribute AdcGroupReplacement (AdcGroup) detailed description

Property	Value
Label	Adc Group Replacement
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_GROUP_REPL_ABORT_RESTART
Enable	false
Range	ADC_GROUP_REPL_ABORT_RESTART ADC_GROUP_REPL_SUSPEND_RESUME

4.8.2.12.7 AdcGroupTriggSrc (AdcGroup)

Type of source event that starts a group conversion. It's possible to select Hw or Sw trigger. In case of Hw trigger the trigger source can be from the CTU or External hardware pins of the controller. In this controller both CTU trigger and external hw pin sources are supported which is selected by the "AdcHwTrigSrc" parameter.

Table 4-87. Attribute AdcGroupTriggSrc (AdcGroup) detailed description

Property	Value
Label	Adc Group Trigger Source
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_TRIGG_SRC_SW
Range	ADC_TRIGG_SRC_HW ADC_TRIGG_SRC_SW

4.8.2.12.8 AdcHwTrigSignal (AdcGroup)

Configures on which edge of the hardware trigger signal the driver should reach, i.e. start the conversion.

Table 4-88. Attribute AdcHwTrigSignal (AdcGroup) detailed description

Property	Value
Label	Adc Group Trigger Signal
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_HW_TRIG_RISING_EDGE
Range	ADC_HW_TRIG_FALLING_EDGE ADC_HW_TRIG_RISING_EDGE

4.8.2.12.9 AdcHwTrigTimer (AdcGroup)

Reload value of the ADC module embedded timer. Isn't used on this version.

Table 4-89. Attribute AdcHwTrigTimer (AdcGroup) detailed description

Property	Value
Label	Adc Group Trigger Timer

Table continues on the next page...

Table 4-89. Attribute AdcHwTrigTimer (AdcGroup) detailed description (continued)

Property	Value
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Enable	false
Invalid	Range <=9223372036854775807 >=0

4.8.2.12.10 AdcNotification (AdcGroup)

Callback function for each group. This function pointer is called everytime when the conversion of this group is completed.

Table 4-90. Attribute AdcNotification (AdcGroup) detailed description

Property	Value
Label	Adc Group Notification
Type	FUNCTION-NAME
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	NULL_PTR

4.8.2.12.11 AdcStreamingBufferMode (AdcGroup)

Select the streaming buffer as linear buffer (i.e. the ADC Driver stops the conversion as soon as the stream buffer is full) or as ring buffer (wraps around if the end of the stream buffer is reached).

Table 4-91. Attribute AdcStreamingBufferMode (AdcGroup) detailed description

Property	Value
Label	Adc Group Streaming Buffer Mode
Type	ENUMERATION
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	ADC_STREAM_BUFFER_LINEAR
Range	ADC_STREAM_BUFFER_CIRCULAR ADC_STREAM_BUFFER_LINEAR

4.8.2.12.12 AdcStreamingNumSamples (AdcGroup)

Number of ADC values to be acquired per channel in streaming access mode.

Table 4-92. Attribute AdcStreamingNumSamples (AdcGroup) detailed description

Property	Value
Label	Adc Group Streaming Number Samples
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	1
Invalid	Range <=255 >=1

4.8.2.12.13 AdcEnableChDisableChGroup (AdcGroup)

Note

If this parameter is enabled, it allows the feature of enabling or disabling a particular channel in the group.

Max.no of Groups with this feature enabled, should be configured are 254 if the configuration parameter AdcEnableChDisableChApi is enabled in NonAutosar container/
 > This is an Implementation Specific Parameter.

Table 4-93. Attribute AdcEnableChDisableChGroup (AdcGroup) detailed description

Property	Value
Label	Adc Group Enable/Disable channels
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.8.2.12.14 AdcMuxDelayNeeded (AdcGroup)

Note

This parameter will be enabled only when both "AdcMuxDelaySupport" and "AdcMuxDelaySupportPerGroup" parameters are enabled and the Group is a SW group.

If enabled, `Adc_StartGroupConversion` Api will be able to change the DSDR values. This is an Implementation Specific Parameter.

Table 4-94. Attribute `AdcMuxDelayNeeded (AdcGroup)` detailed description

Property	Value
Label	Adc Mux Delay Needed
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.8.2.12.15 `AdcWithoutInterrupts (AdcGroup)`

Enable/ Disable the occurring of ADC Interrupts and Reading of the group conversion results periodically without interrupts. A) When this parameter is enabled, interrupts are disabled . The conversion will run without software intervention (no interrupt generated anymore) and application can read the results by calling `Adc_ReadGroup()`. B) When this parameter is enabled, the result buffer is no longer to be used to read the results as the results will be directly read from HW registers. When this parameter is Disabled, normal functionlaity shall be executed. Note: This is an Implementation Specific Parameter.

Table 4-95. Attribute `AdcWithoutInterrupts (AdcGroup)` detailed description

Property	Value
Label	Adc Group Without Interrupts
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.8.2.12.16 `AdcMultipleHardwareTriggerGroup (AdcGroup)`

If is checked that means that this group is part of the MHT groups subset. MHT - Multiple Hardware Trigger. That meaning a subset of groups can be active at a given time as Hardware Triggered groups. To use this feature it is required that every MHT marked group share the same settings on several parameters. The list of parameters that can be different is: - Group Id - actually it should be different. - Group Priority - it can have any value in the domain, it doesnt matter. The priority will be based on the selected CTU trigger priority. - Group Notification - can be different. - Group Buffer Pointer -

recommended to be different to know which value from which group/channel comes. - AdcHwTrigSrc - should be different for every group. If two or more MHT groups share the same trigger source it won't work. Each such a grup (MHT) should have only and only one ADC channel! These ADC channel should be unique across these MHT groups (actually they should be unique across the MHT subset runtime!).

Table 4-96. Attribute AdcMultipleHardwareTriggerGroup (AdcGroup) detailed description

Property	Value
Label	Adc MHT Group
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.8.2.12.17 Form AdcGroupDefinition

Is included by form : [Form AdcGroup](#)

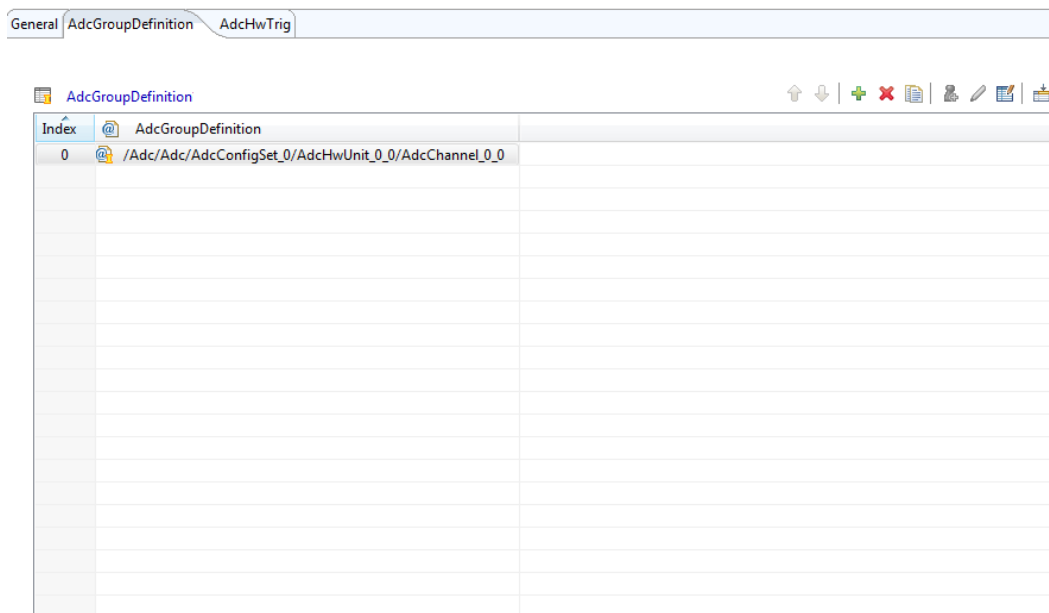


Figure 4-14. Tresos Plugin snapshot for AdcGroupDefinition form.

4.8.2.12.17.1 AdcGroupDefinition (AdcGroupDefinition)

Assignment of channels to a AdcGroups. For each AdcChannel that should belong to the group, a reference needs to be defined.

Table 4-97. Attribute AdcGroupDefinition (AdcGroupDefinition) detailed description

Property	Value
Type	REFERENCE
Origin	AUTOSAR_ECUC

4.8.2.12.18 Form AdcGroupConversionConfiguration

Configure the Sampling and Conversion TimeGroup.

Is included by form : [Form AdcGroup](#)

The screenshot shows the 'AdcGroupConversionConfiguration' form in the Tresos Plugin. It contains three configuration items:

- Adc Group Latch Duration***: A dropdown menu set to 'ADC_HALF_CLOCK_CYCLE'.
- Adc Group Comparison Duration***: A dropdown menu set to 'ADC_TWO_LATCH_CYCLE'.
- Adc Group Sampling Duration (3 -> 255)***: A text input field set to '8'.

Figure 4-15. Tresos Plugin snapshot for AdcGroupConversionConfiguration form.

4.8.2.12.18.1 AdcLatchingDuration (AdcGroupConversionConfiguration)

Select the Bit Latching Phase Duration. Usually this value should be ADC_ONE_CLOCK_CYCLE.

Table 4-98. Attribute AdcLatchingDuration (AdcGroupConversionConfiguration) detailed description

Property	Value
Label	Adc Group Latch Duration
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_HALF_CLOCK_CYCLE
Range	ADC_HALF_CLOCK_CYCLE ADC_ONE_CLOCK_CYCLE

4.8.2.12.18.2 AdcComparisonDuration (AdcGroupConversionConfiguration)

Select the Comparison Duration. Usually this value should be ADC_THREE_LATCH_CYCLE.

Table 4-99. Attribute AdcComparisonDuration (AdcGroupConversionConfiguration) detailed description

Property	Value
Label	Adc Group Comparison Duration
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	ADC_TWO_LATCH_CYCLE
Range	ADC_ONE_LATCH_CYCLE ADC_TWO_LATCH_CYCLE ADC_THREE_LATCH_CYCLE ADC_FOUR_LATCH_CYCLE

4.8.2.12.18.3 AdcSamplingDuration (AdcGroupConversionConfiguration)

Selects value to be written into bit-field INPSAMP of the CTR register. The sampling duration component (T_{sample}) of the total conversion time $\{T_{\text{sample}} + T_{\text{eval}} + N_{\text{delay}} * nT_{\text{ck}}\}$ is given by:

$$T_{\text{sample}} = (\text{INPSAMP} - N_{\text{delay}}) * T_{\text{ck}}$$

The timing refers to T_{ck} , where $F_{\text{ck}} = (1/2 * \text{ADC peripheral set clock})$

$N_{\text{delay}} = 0.5$ (for $\text{INPSAMP} \leq 6$); $N_{\text{delay}} = 1$ (for $\text{INPSAMP} > 6$). INPSAMPLE must be greater than or equal to 8 (hardware requirement)

$T_{\text{eval}} = n * T_{\text{biteval}}$; where: 'n' = Channel resolution (10 or 12 bits); $T_{\text{biteval}} = \text{INPCMP} * T_{\text{ck}}$ if $\text{INPCMP} \geq 1$.

Only for mpc560[2|3|4][B|C], spc560[B|C][40x|50x] and mpc560[5|6|7]B, spc560B[54|60|64]x if $\text{INPCMP} = 0$, $T_{\text{biteval}} = 4 * T_{\text{ck}}$.

The hardware imposes the following restrictions on the choice of INPSAMP:

Table 4-100. Detailed description.

Subderivative type	mpc560[1 2]D, spc560D[30 40]	mpc560[2 3 4][B C], spc560[B C][40x 50x]	mpc560[5 6 7]B, spc560B[54 60 64]x	mpc560[5 6 7]B, spc560B[54 60 64]x
Adc Unit	ADC_1	ADC_0	ADC_0	ADC_1
Min T_{sample}	600ns	500ns	500ns	600ns
Min INPSAMP	8	3	3	8

Table 4-101. Detailed description.

Subderivative type	mpc560[1 2]D, spc560D[30 40]	mpc560[2 3 4][B C], spc560[B C][40x 50x]	mpc560[5 6 7]B, spc560B[54 60 64]x	mpc560[5 6 7]B, spc560B[54 60 64]x
Adc Unit	ADC_1	ADC_0	ADC_0	ADC_1
Min Tsample	500ns	500ns	500ns	500ns
Min INPSAMP	8	3	3	8

**Table 4-102. Attribute AdcSamplingDuration
(AdcGroupConversionConfiguration) detailed description**

Property	Value
Label	Adc Group Sampling Duration
Type	INTEGER
Origin	Custom
Symbolic Name	false
Invalid	Range >=3 <=255

4.8.2.12.19 Form AdcHwTrig

This container contains the Hardware trigger source configured for the group.

Is included by form : [Form AdcGroup](#)

Figure 4-16. Tresos Plugin snapshot for AdcHwTrig form.

4.8.2.12.19.1 AdcHwTrigSrc (AdcHwTrig)

Note

Select the TIMER / PIT Trigger source channels for the ADC trigger inputs. Available Trigger Sources from the CTU modules are:

MPC5601/2D:

eMIOS_A_Ch0 to eMIOS_A_CH22, PIT3, eMIOS_A_CH24 to eMIOS_A_CH27

MPC5602/3/4B/C:

eMIOS_A_Ch0 to eMIOS_A_CH22, PIT3, eMIOS_A_CH24, eMIOS_B_CH0 to eMIOS_B_CH22, eMIOS_B_CH24

MPC5605/6/7B:

eMIOS_A_Ch0 to eMIOS_A_CH22, PIT3, eMIOS_A_CH24 to eMIOS_A_CH31, eMIOS_B_CH0 to eMIOS_B_CH22, PIT7, eMIOS_B_CH24 to eMIOS_B_CH31

Available Trigger Sources without the CTU module are: PIT2 for ADC0 PIT6 for ADC1 Hardware Priority for the "eMIOS_A_Ch0" is highest while for the "eMIOS_B_Ch24" is lowest. Priority configured for the group with "ADC_TRIGG_SRC_HW" (Hardware trigger source) should be always greater then the "ADC_TRIGG_SRC_SW" (Software trigger source) configured group. This is an Implementation Specific Parameter.

Table 4-103. Attribute AdcHwTrigSrc (AdcHwTrig) detailed description

Property	Value
Type	ENUMERATION
Origin	Custom
Symbolic Name	false
Default	eMIOS_A_Ch0

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number UM13ADCASR4.0 Rev0003R1.0.1
Revision 1.2